

Politecnico di Torino

Politecnico di Torino

Corso di Laurea Magistrale in Ingegneria Informatica

A.A. 2025/2026

Sessione di laurea Dicembre 2026

GroundOne: un System One specialista per la verifica della groundedness nella RAG locale

Fine-tuning di modelli linguistici compatti come verificatori calibrati a singolo passaggio su Apple Silicon

Relatore: Prof. Paolo Garza

Candidato: Francesco Pio De Dominicis

Sommario

La retrieval-augmented generation (RAG) è ormai il modo standard per dare ai modelli linguistici accesso a documenti privati, e oggi l'intera pipeline può essere eseguita sul dispositivo dell'utente con hardware di fascia consumer, a vantaggio della privacy, dell'uso offline e dei costi. Il recupero dei documenti, tuttavia, non elimina le allucinazioni: le risposte generate contengono ancora frasi che il contesto recuperato non supporta o che lo contraddicono. Individuarle è un prerequisito per assistenti locali affidabili e, poiché il controllo va eseguito su ogni risposta, deve trovare posto sulla stessa macchina, dalle risorse limitate, che ospita il generatore.

La verifica segue oggi due paradigmi. I verificatori basati su encoder, come AlignScore, MiniCheck e HHEM, sono piccoli e veloci, ma restituiscono un unico punteggio fisso per ogni coppia di documento e frase. I giudici generativi sono flessibili, ma scrivono i verdetti token per token, per cui il loro costo cresce con la quantità di testo prodotto; i giudici piccoli violano spesso il formato di output, e la confidenza che esprimono è raramente utilizzabile come probabilità. Di recente è emersa una terza famiglia: i **modelli System One**, come il proprietario Jev e i modelli aperti SemIf e Rizzo-Flow, che rispondono a domande chiuse in un unico forward pass leggendo dai logit la probabilità di ciascuna opzione di risposta. Le evidenze a loro favore provengono per lo più dagli stessi autori, e le implementazioni aperte sono generaliste. Non era ancora stato studiato in modo controllato se un System One compatto, specializzato in un unico task, possa eguagliare i giudici generativi e gli encoder dedicati, da dove derivi il suo vantaggio e se le sue probabilità siano utilizzabili, il tutto su un portatile.

Questa tesi progetta, addestra e valuta **GroundOne**, un System One specializzato in un'unica decisione: stabilire, per ogni frase di una risposta, se il documento recuperato la supporta, la contraddice o non la menziona. Ogni frase della risposta è seguita da un marker, e i verdetti di tutte le K frasi si leggono dai logit di tre lettere di risposta in un unico forward pass, senza generare testo. Modelli Qwen2.5 da 0.5B e 3B parametri sono sottoposti a fine-tuning con LoRA sulle risposte di RAGTruth annotate manualmente, con una loss pesata per classe e ristretta alle tre lettere; una temperatura e un bias stimati sul dev set calibrano le probabilità, e una soglia scelta sugli stessi dati le traduce in etichette coerenti e in segnalazioni

secondo la policy strict o lenient. Tutto, dall'addestramento alla messa in servizio, gira con MLX su un portatile Apple M4 con 16 GB di memoria. La valutazione usa tre test set costruiti e congelati prima dell'addestramento (la partizione di test di RAGTruth, LLM-AggreFact e VitaminC) e confronta GroundOne con indizi che non leggono mai il documento, con i modelli non addestrati, con giudici generativi della stessa dimensione, con e senza probabilità, con un System One generalista e con encoder specializzati, riportando intervalli calcolati con un bootstrap a cluster.

Sul test set di RAGTruth GroundOne raggiunge una balanced accuracy a tre classi di 0.769 con 0.5B parametri e di 0.832 con 3B, contro 0.37 e 0.44 dei modelli non addestrati; la sua AUROC (0.905 e 0.940) supera quella degli encoder (circa 0.82), e il modello supera nettamente sia un System One generalista otto volte più grande sia un decision model generalista della stessa dimensione (0.657), che a sua volta è molto più forte sul test fuori dominio VitaminC. Su LLM-AggreFact il 3B è alla pari con MiniCheck-RoBERTa (74.8 contro 74.0, differenza non significativa), sopra il modello 7B zero-shot pubblicato e sotto il System One chiuso Jev, misurato in modo indipendente (78.6). Per risposte di quattordici o più frasi GroundOne è circa due volte più veloce di un giudice sintetico e sei volte più veloce di uno verboso della stessa dimensione, con un output sempre valido; consuma inoltre meno energia (5.5 e 33 joule per risposta), sebbene con un fattore inferiore a quello osservato sul tempo, perché un prefill lungo assorbe più potenza del decoding. Quando si chiede loro lo stesso output, i giudici generativi forniscono probabilità che distinguono a stento le decisioni corrette da quelle errate, mentre la calibrazione porta l'Expected Calibration Error di GroundOne sul test set a 0.019. A parità di pesi, restringere la decisione alle tre lettere è più accurato che prendere il token che il modello genererebbe, di 0.25 per il 0.5B e di 0.07 per il 3B. I limiti sono altrettanto chiari: il trasferimento a un dominio diverso è debole, le allucinazioni nascoste in frasi che per il resto ricalcano la fonte restano difficili da individuare, e la ricetta di addestramento non è del tutto robusta rispetto alla scelta del seed.

Ringraziamenti

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodo sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

Indice

Elenco delle figure VIII

1 Introduzione 1
- 1.1 Motivazione . . . 1
- 1.2 Modelli System One . . . 1
- 1.3 Domande di ricerca . . . 2
- 1.4 Contributi . . . 3
- 1.5 Struttura della tesi . . . 4

2 Fondamenti e stato dell'arte 5
- 2.1 Inferenza nei transformer: prefill, decoding e KV cache . . . 5
- 2.2 Inferenza sul dispositivo dell'utente . . . 6
- 2.3 Modelli System One . . . 7
- 2.4 Retrieval-augmented generation e allucinazioni . . . 9
- 2.5 Verifica della groundedness . . . 9
- 2.6 Calibrazione e selective prediction . . . 10
- 2.7 Parameter-efficient fine-tuning . . . 11
- 2.8 Lacuna nella letteratura . . . 11

3 GroundOne: metodo e implementazione 12
- 3.1 Task e decisione . . . 12
- 3.2 Tutti i verdetti in un solo forward pass . . . 13
- 3.3 Obiettivo di addestramento . . . 14
- 3.4 Calibrazione . . . 15
- 3.5 Software . . . 16

4 Dati 18
- 4.1 Task ed etichette . . . 18
- 4.2 Dataset di partenza . . . 19
- 4.3 Costruzione . . . 20
- 4.4 Il dataset risultante . . . 21

4.5 Affidabilità dei test set . . . 23
4.6 Analisi esplorativa . . . 24
4.7 Controllo del leakage . . . 26

5 Impostazione sperimentale 27
5.1 Modelli . . . 27
5.2 Baseline . . . 28
5.3 Metriche . . . 30
5.4 Protocollo statistico . . . 31
5.5 Latenza, energia e hardware . . . 31

6 Risultati 33
6.1 Addestramento . . . 33
6.2 Variabilità tra seed . . . 33
6.3 Accuratezza sul test set RAG (T1) . . . 35
6.4 Confronto con le baseline . . . 37
6.5 Trasferimento a un dominio diverso (T3) . . . 39
6.6 Readout e generazione a confronto . . . 39
6.7 Calibrazione e selective prediction . . . 40
6.8 Probabilità a parità di output . . . 42
6.9 Latenza in funzione di K . . . 43
6.10 Energia per verifica . . . 44
6.11 Analisi degli errori . . . 45

7 Discussione, limiti e minacce alla validità 47
7.1 Quando conviene un System One specialista . . . 47
7.2 Limiti . . . 49
7.3 Minacce alla validità . . . 50

8 Conclusioni e sviluppi futuri 52
8.1 Sintesi dei risultati . . . 52
8.2 Sviluppi futuri . . . 53

A Output di ogni sistema su una stessa risposta 54
A.1 Verdetti frase per frase . . . 55
A.2 Output grezzi . . . 56

Bibliografia 58

Elenco delle figure

4.1 Distribuzione delle classi delle frasi etichettate in ciascuno split. RAGTruth (train, dev e T1) è supportato per circa l'88–91 %; T3 è campionato a 1,000 coppie per classe. 22

4.2 Distribuzione di K, il numero di frasi valutate per risposta, nel training set e in T1. Le linee tratteggiate indicano le fasce usate per riportare i risultati in funzione di K. 23

4.3 Quota di frasi contraddette o non menzionate in base al modello che ha scritto la risposta, nel training set e in T1. 25

4.4 Sovrapposizione lessicale tra una frase e il suo documento, per classe, nel training set, in T1 e in T3 (media tra parentesi). 26

4.5 Quota di frasi contraddette o non menzionate in base alla posizione nella risposta. 26

6.1 Balanced accuracy a tre classi sul dev set durante l'addestramento del modello 0.5B, complessiva (a sinistra) e per task (a destra). La linea punteggiata indica il livello casuale. Il 3B mostra lo stesso andamento sul suo sottoinsieme di 200 risposte del dev set. 34

6.2 Balanced accuracy a tre classi sul dev set del modello 0.5B durante l'addestramento, con la stessa procedura e tre seed. Due esecuzioni convergono allo stesso livello; la terza è collassata su un predittore costante ed è stata interrotta. 35

6.3 T1: AUROC di P(supported) per ogni sistema (a sinistra) e balanced accuracy a tre classi per i sistemi con output a tre classi (a destra). Rizzo-Flow è valutato su 300 risposte e il 3B non addestrato su 600; tutti gli altri sistemi, compreso il decision model generalista d1-3B, sull'intero test set. 37

6.4 Controllo B2 sull'intero T1: con gli stessi pesi e lo stesso forward pass, il verdetto è letto dalle tre lettere di risposta (GroundOne) oppure preso come il token che emetterebbe la generazione greedy (gli errori di formato sono contati come errori). 40

6.5 Reliability diagram di P(supported) del 0.5B prima (grezza) e dopo la calibrazione sul dev set, per il dev set, T1 e T3. I punti corrispondono a intervalli con almeno 20 frasi. 41
6.6 Curve risk–coverage della decisione binaria del 0.5B con probabilità calibrate: tasso di errore al crescere delle frasi meno sicure che vengono rinviate. 41
6.7 Confronto a parità di output sulle 200 risposte di latenza: giudici generativi della stessa dimensione a cui si chiede l'output di GroundOne, con probabilità lette dai logit della lettera generata oppure scritte in JSON; tutti i sistemi sono calibrati sul dev set con la stessa mappa. Solo frasi con un verdetto valido. 42
6.8 Tempo mediano per risposta in funzione di K sulle stesse 200 risposte di T1 (MLX in-process, una risposta alla volta, M4 con 16 GB), per GroundOne, il System One con una domanda per frase e i giudici generativi sintetico e verboso. Scala logaritmica. 44
6.9 Sul sottoinsieme di latenza: balanced accuracy a tre classi (gli errori di formato sono contati come errori) e quota di frasi il cui verdetto non è stato possibile interpretare. 44
6.10 Energia per risposta verificata su 100 risposte di latenza (powermetrics, CPU, GPU e Neural Engine, al netto della potenza a riposo), con la potenza netta media durante l'esecuzione. 45

Capitolo 1

Introduzione

1.1 Motivazione

La retrieval-augmented generation (RAG) è ormai l'approccio standard per far rispondere un modello linguistico a domande su documenti che non ha mai visto in addestramento: un retriever seleziona i passaggi pertinenti e il modello formula la risposta avendoli nel proprio contesto [1]. Dato che modelli aperti compatti e runtime efficienti girano ormai su hardware consumer [2, 3], l'intera pipeline può restare sul dispositivo dell'utente. Questo conta ogni volta che i documenti sono riservati, che l'assistente deve funzionare senza connessione di rete o che il suo comportamento non deve cambiare in seguito a un aggiornamento del fornitore.

Il recupero dei documenti, però, non elimina le allucinazioni. Le risposte contengono comunque frasi che i passaggi recuperati non supportano, o che li contraddicono [4, 5], e non si può pretendere che l'utente confronti ogni frase con la fonte. Un assistente locale affidabile ha quindi bisogno di un verificatore che controlli ogni risposta, frase per frase, sulla stessa macchina che l'ha generata. Un verificatore di questo tipo si contende con il generatore la stessa memoria, lo stesso tempo e la stessa batteria. La soluzione più comune, cioè chiedere a un modello linguistico di giudicare la risposta, è costosa proprio dove un laptop è più debole: un giudice scrive i propri verdetti token per token, e su hardware locale generare un token costa molto più che leggerlo. I giudici piccoli, inoltre, non rispettano il formato di output richiesto, e la confidenza che dichiarano è raramente utilizzabile come probabilità.

1.2 Modelli System One

Un modo diverso di usare un modello linguistico per prendere decisioni ha attirato di recente l'attenzione con il nome di *modelli System One* [6]. Invece di generare una

risposta, un modello di questo tipo legge una sola volta la domanda e il contesto e restituisce una probabilità per ciascuna risposta ammissibile, per esempio leggendo i logit delle lettere delle opzioni alla fine del prompt [7, 8]. Non viene generato alcun testo, l'output è sempre valido e molte domande sullo stesso contesto possono condividere un unico passaggio su di esso. L'idea si presta bene alla verifica: una risposta di K frasi pone K domande chiuse sullo stesso documento, e alla probabilità associata a ogni frase si possono applicare una soglia e una calibrazione.

Le evidenze a favore dei modelli System One, tuttavia, provengono per lo più dai loro stessi autori, e le implementazioni aperte sono sistemi generalisti in grado di rispondere a qualunque domanda tipizzata. Restano aperte diverse questioni: se l'approccio convenga per un task specifico, come si collochi un modello specializzato in quel task rispetto a uno generalista, ai giudici generativi e ai verificatori dedicati, e da dove derivi davvero l'eventuale vantaggio.

1.3 Domande di ricerca

Questa tesi progetta, addestra e valuta GroundOne, un System One compatto specializzato in un'unica decisione: stabilire, per ogni frase di una risposta RAG, se il documento recuperato la supporta, la contraddice o non la menziona. Il lavoro è guidato da cinque domande di ricerca.

RQ1, specializzazione. Un System One da 0.5B o 3B parametri, sottoposto a fine-tuning, può eguagliare modelli zero-shot più grandi e gli encoder specializzati nella valutazione della groundedness a livello di frase, cioè dell'aderenza di ciascuna frase al documento?

RQ2, readout contro generazione. Che cosa apporta il meccanismo System One, ossia il readout della decisione dai logit, a parità di dimensione del modello, rispetto a giudici generativi che scrivono i propri verdetti e allo stesso modello che decide in base al token che genererebbe, e come cresce la differenza all'aumentare del numero di decisioni per risposta?

RQ3, calibrazione. Le probabilità sono utilizzabili come misura di confidenza, per fissare soglie e per rinviare le decisioni incerte, e come si confrontano con quelle che un giudice generativo è in grado di fornire?

RQ4, costo sul dispositivo. Il verificatore rientra nel budget di memoria, tempo ed energia di un laptop da 16 GB, e come si confronta il suo costo con quello di un giudice eseguito sulla stessa macchina?

RQ5, oltre il punteggio binario. Che cosa offre una decisione a tre classi per ogni frase che un verificatore binario non offre?

1.4 Contributi

La tesi offre quattro contributi.

1. **GroundOne**, un System One specialista e aperto per la verifica della groundedness a livello di frase, eseguito interamente su un laptop. Grazie a un marker inserito dopo ogni frase, legge tutti i verdetti di una risposta in un solo forward pass; è rilasciato come pacchetto installabile, con un'interfaccia a riga di comando e un'API HTTP locale che comprende anche un'interfaccia compatibile con i client System One esistenti.

2. **Un dataset costruito attorno al task** a partire da annotazioni umane pubbliche, con tre test set validati senza nuove annotazioni e congelati prima di addestrare qualunque modello, e con il leakage controllato a livello di documento.

3. **Una scomposizione controllata dei guadagni.** Il confronto isola il contributo della specializzazione, del readout ristretto e della dimensione del modello rispetto a indizi che non verificano mai una frase, modelli non addestrati, giudici generativi con e senza probabilità, un System One generalista ed encoder specializzati, tutti sullo stesso hardware e con intervalli di confidenza ottenuti tramite cluster bootstrap.

4. **Uno studio di calibrazione, tempo ed energia**: come le probabilità possano essere calibrate e tradotte in etichette coerenti, quanto aiutino a rinviare le decisioni incerte e come tempo ed energia per verifica crescano con la lunghezza della risposta.

In sintesi, i risultati mostrano che la specializzazione funziona nel dominio RAG. Sul test set RAGTruth GroundOne raggiunge una balanced accuracy a tre classi di 0.769 con 0.5B parametri e di 0.832 con 3B: più degli encoder specializzati, molto più di un System One generalista otto volte più grande e più di un decision model generalista della stessa dimensione. Su LLM-AggreFact, inoltre, il modello da 3B è alla pari con il miglior encoder di piccole dimensioni. Sulle risposte lunghe la verifica è circa due volte più veloce di un giudice sintetico e sei volte più veloce di un giudice verboso della stessa dimensione, con output sempre valido, probabilità molto migliori e un minore consumo di energia. La sola restrizione del readout alle risposte ammissibili basta ad aumentare l'accuratezza. Altrettanto chiari sono i limiti: il trasferimento a un dominio diverso è debole, e lì il modello generalista risulta migliore; le allucinazioni nascoste in frasi che per il resto ricopiano la fonte restano difficili da individuare; la ricetta di addestramento, infine, non è del tutto robusta rispetto alla scelta del seed.

1.5 Struttura della tesi

Il Capitolo 2 passa in rassegna l'inferenza nei transformer, l'inferenza locale, i modelli System One, le allucinazioni nella retrieval-augmented generation, i metodi di verifica e la calibrazione. Il Capitolo 3 descrive GroundOne: la decisione, il readout in un solo passaggio, l'obiettivo di addestramento, la calibrazione e l'output esposto dal servizio. Il Capitolo 4 definisce le etichette e descrive come sono stati costruiti, controllati e congelati il training set e i test set. Il Capitolo 5 presenta i modelli addestrati, le baseline, le metriche e il protocollo di misura. Il Capitolo 6 riporta i risultati, il Capitolo 7 ne discute il significato e i limiti e il Capitolo 8 trae le conclusioni. L'Appendice A mostra l'output di tutti i sistemi su una stessa risposta.

Capitolo 2

Fondamenti e stato dell'arte

Questo capitolo introduce le idee su cui si basa GroundOne e i lavori con cui viene confrontato. Si parte da come un transformer impiega il proprio tempo durante l'inferenza, perché il vantaggio di un System One ne discende direttamente, e da ciò che cambia quando l'inferenza avviene sul dispositivo dell'utente. Si passano poi in rassegna i modelli System One, il problema delle allucinazioni nella retrieval-augmented generation (RAG) e i metodi che verificano le risposte rispetto alle fonti; il capitolo si chiude con la calibrazione, il parameter-efficient fine-tuning e la lacuna che questa tesi intende colmare.

2.1 Inferenza nei transformer: prefill, decoding e KV cache

Un modello linguistico decoder-only elabora l'input in due fasi. Nella fase di *prefill* l'intero prompt viene letto in parallelo: ogni strato calcola, per ciascun token di input, le chiavi e i valori che i token successivi useranno nel meccanismo di attenzione, e li memorizza nella KV cache. Nella fase di *decoding* il modello produce l'output un token alla volta; ogni nuovo token consulta la cache attraverso l'attenzione e la estende di una posizione [9]. Sull'hardware le due fasi si comportano in modo molto diverso. Il prefill esegue molto calcolo su molti token contemporaneamente ed è limitato dalla potenza di calcolo aritmetico. Il decoding esegue poco calcolo a ogni passo, ma per ogni token generato deve leggere tutti i pesi e la cache, che nel frattempo cresce: con batch piccoli è quindi limitato dalla banda di memoria [9, 10]. Ne segue che un token generato costa molto più tempo di un token del prompt, e su un singolo dispositivo la differenza arriva a un ordine di grandezza. I sistemi di serving sfruttano questa distinzione in modi diversi, per esempio gestendo la cache a pagine per accogliere più richieste concorrenti [11] oppure eseguendo le due fasi su macchine separate, ciascuna adatta alla propria [10].

La conseguenza pratica per la verifica è semplice. Un giudice che scrive il proprio verdetto paga un prefill per il documento e la risposta, poi un passo di decoding per ogni token che scrive, che si tratti di una sola lettera o di un paragrafo di ragionamento. Un modello che legge la decisione dai logit calcolati durante il prefill paga soltanto il primo costo. La KV cache offre una seconda leva: quando si pongono molte domande sullo stesso contesto, questo può essere elaborato una volta sola e poi riutilizzato, così che ogni domanda in più costi soltanto i suoi pochi token.

2.2 Inferenza sul dispositivo dell'utente

I modelli linguistici si usano per lo più tramite servizi remoti, che ricevono l'input dell'utente attraverso la rete ed eseguono il modello in un datacenter. Eseguire il modello direttamente sul dispositivo dell'utente cambia contemporaneamente diverse proprietà. I documenti non lasciano mai la macchina, aspetto importante per dati riservati o soggetti a regolamentazione; il sistema funziona senza rete e senza dipendere dalla disponibilità di un fornitore; il costo è quello dell'hardware e dell'energia che consuma, non un prezzo per token; i pesi, e quindi il comportamento, non cambiano se non è l'utente a cambiarli. Un modello locale espone inoltre i propri logit, che le interfacce remote spesso nascondono o limitano e di cui un System One e la sua calibrazione hanno bisogno. Il prezzo da pagare è la capacità del modello: un laptop può ospitare soltanto modelli piccoli, e deve farlo accanto a tutto ciò che vi è già in esecuzione.

Tre proprietà dell'hardware stabiliscono che cosa sia fattibile. La capacità di memoria determina quali modelli possono essere caricati: un modello da 7B in mezza precisione richiede più di 14 GB, oltre la portata della maggior parte dei dispositivi personali; per questo le installazioni locali ricorrono a pesi quantizzati, a modelli più piccoli oppure, nei sistemi di ricerca, al caricamento in streaming dei pesi dalla memoria flash [12]. La banda di memoria limita la velocità del decoding, perché ogni token generato legge tutti i pesi: sul laptop usato in questa tesi, un Apple M4 con 120 GB/s di banda e 16 GB di memoria unificata condivisa tra CPU e GPU [13], questo limite vale pochi millisecondi per token per un modello da 0.5B a 8 bit e circa dieci volte tanto per un modello da 3B. La potenza di calcolo, infine, limita il prefill, che sulla GPU di un laptop è relativamente rapido.

Il software per l'inferenza locale è maturato rapidamente. llama.cpp [2] offre inferenza senza dipendenze esterne, con quantizzazione intera e backend per la maggior parte degli acceleratori consumer, e su di esso si basano molti strumenti di distribuzione. MLX [3], insieme al suo pacchetto per modelli linguistici [14], è un framework per array progettato per la memoria unificata di Apple silicon e supporta quantizzazione e fine-tuning LoRA direttamente sul dispositivo. I produttori di

piattaforme distribuiscono ormai i propri modelli on-device: il foundation model che Apple impiega sui propri dispositivi è un modello denso di circa tre miliardi di parametri, quantizzato a meno di quattro bit per peso e specializzato per ogni funzionalità con adapter low-rank di rango 16 su tutti gli strati lineari [15]. È lo stesso schema, alla stessa scala, che segue GroundOne.

L'energia è l'ultima dimensione dell'inferenza locale. Misure su hardware da datacenter mostrano che l'energia di una richiesta cresce con il numero di token generati e dipende fortemente dalla dimensione del modello e dal task [16, 17]; inoltre, la distinzione tra un prefill limitato dal calcolo e un decoding limitato dalla banda implica che anche la potenza assorbita dalle due fasi sia diversa [10]. Su un dispositivo a batteria l'energia per verifica è quindi una proprietà del metodo e non soltanto dell'hardware, e il Capitolo 6 la misura.

2.3 Modelli System One

Il termine *System One model* è stato introdotto da TypeSafe con Jev, un modello offerto come servizio remoto che risponde a domande chiuse su uno stato dato con output tipizzati, come un booleano, una scelta tra opzioni o un punteggio, senza generare testo [6]. Il nome richiama il pensiero veloce e intuitivo descritto da Kahneman, contrapposto al ragionamento deliberato. Il fornitore dichiara latenze tra le decine e le centinaia di millisecondi e forti risparmi rispetto ai giudici generativi, ma queste cifre restano dichiarazioni e l'architettura non è resa pubblica.

Sono seguite implementazioni aperte, che si distinguono per il modo in cui producono le probabilità. La famiglia più diffusa conserva la head del modello linguistico e legge i logit delle lettere delle opzioni nella posizione della risposta, dopo un prompt che presenta la domanda come scelta multipla. SemIf applica questo readout a un modello congelato da 4B e riporta che 21 criteri relativi a un documento ottengono risposta in circa un secondo, contro più di cinque secondi della migliore baseline generativa capace di produrre JSON valido [7]. Rizzo-Flow applica il fine-tuning LoRA a un modello da 4B su decisioni tipizzate, espone la stessa interfaccia di Jev e riporta miglioramenti di accuratezza e calibrazione dopo il fine-tuning [8]; è il System One generalista usato come baseline in questa tesi. Una seconda famiglia codifica separatamente lo stato e ciascuna risposta candidata e ne confronta gli embedding: i Contrastive Language Models sono molto veloci quando i candidati sono numerosi e riutilizzabili, ma nel confronto riportato dagli stessi autori concordano con le decisioni di Jev molto meno di un modello che fa interagire stato e candidato [18]. Una terza famiglia usa piccoli encoder con una head decisionale dedicata. Laya, per esempio, assegna un punteggio a ogni opzione sul relativo token di maschera di un encoder ModernBERT da circa 400M

parametri; le domande di una richiesta sono valutate in un'unica chiamata a batch, ma ciascuna viene codificata con una propria copia del contesto [19, 20].

Le evidenze a supporto di questi sistemi sono disomogenee. La maggior parte proviene dagli autori stessi, spesso su una sola macchina e su un solo benchmark. Le valutazioni indipendenti sono recenti e meno favorevoli ai piccoli modelli aperti. Una valutazione di Jev su 37 dataset trova ben calibrate le sue probabilità sulle scelte, mentre quelle binarie, pur ordinando bene gli esempi, risultano mal posizionate rispetto a una soglia fissa di 0.5, e riporta una balanced accuracy media di 78.6 su LLM-AggreFact [21]; una riproduzione indipendente di Laya osserva una calibrazione diversa da quella dichiarata e una forte dipendenza da una temperatura stimata sui dati di destinazione [22]; benchmark contrastivi con protocolli chiari collocano i piccoli modelli aperti molto al di sotto di Jev [23]; infine, è stato mostrato che i decision model tipizzati seguono l'etichetta di un'opzione anziché la sua definizione quando le due sono in conflitto [20]. Nel 2026 l'hub di Hugging Face ha introdotto un tag dedicato a questi *decision model* [24], che a ottobre 2026 raccoglieva già circa 1,500 modelli pubblicati da circa 600 autori, di qualità molto variabile.

A poche settimane dal rilascio di Jev, organizzazioni affermate hanno pubblicato i propri decision model aperti. Clef di Cloudflare aggiunge a un decoder una head decisionale dedicata e risponde a tutte le domande in un solo passaggio sullo stato, l'architettura che per Jev era stata soltanto ipotizzata [25]; d1-3B di Liquid AI legge lettere di risposta ristrette da un modello da 3B il cui stato è condiviso da tutte le domande [26]; Perplexity applica il fine-tuning a un modello da 27B con una head sulle lettere [27]. Una leaderboard della comunità, il Decision Index, valuta questi modelli su una batteria comune di benchmark pubblici che comprende RAGTruth a livello di risposta intera [28]. I risultati tracciano una linea netta: i decision model generalisti con meno di un miliardo di parametri superano a stento la baseline banale su RAGTruth, e solo modelli da decine di miliardi di parametri individuano in modo affidabile le risposte allucinate. Gli studi su Jev aggiungono che le sue probabilità sono calibrate se aggregate su molti task ma non sul singolo task, che soglie e temperature vanno stimate sul carico di lavoro di destinazione [29, 30] e che scegliere la soglia tramite un limite inferiore di confidenza rende più sicuro mettere in produzione un giudice che restituisce soltanto decisioni [30]. Anche i runtime si sono adeguati: llama.cpp supporta ora nativamente l'interfaccia System One [2]. Nessuno di questi modelli, tuttavia, è specializzato nella groundedness, cioè nell'aderenza della risposta al documento, e nessuno riporta risultati di verifica a tre classi a livello di frase sui benchmark usati in questa tesi.

Il readout sulle lettere è stato studiato anche al di fuori della letteratura sui System One. Nella valutazione a scelta multipla, la probabilità del primo token di risposta e la risposta che un modello instruction-tuned scrive effettivamente sono spesso in disaccordo [31], e il readout privilegia particolari identificatori di

opzione, un bias che si può stimare e rimuovere [32]. Entrambi i risultati riguardano modelli zero-shot e suggeriscono che un readout sulle lettere usato come decisione vada addestrato, calibrato o entrambe le cose. Nessun System One aperto è stato specializzato nella verifica della groundedness e confrontato, in condizioni controllate e sullo stesso hardware, con giudici generativi, verificatori specializzati e un decision model generalista della stessa dimensione.

2.4 Retrieval-augmented generation e allucinazioni

La retrieval-augmented generation dà a un modello linguistico accesso a documenti su cui non è stato addestrato: un retriever seleziona i passaggi pertinenti alla domanda e il modello scrive la risposta con questi passaggi nel contesto [1]. È l'approccio standard per costruire assistenti su documenti privati e può girare interamente su una macchina locale. Non elimina però le allucinazioni. Le rassegne della letteratura distinguono gli errori di *fattualità* (factuality), rispetto alla conoscenza del mondo, dagli errori di *fedeltà* (faithfulness), rispetto all'input o al contesto forniti [5]; una risposta RAG può essere infedele proprio ai passaggi che ha ricevuto, contraddicendoli o aggiungendo informazioni che non contengono. RAGTruth documenta quanto ciò sia frequente: raccoglie circa 18,000 risposte scritte da sei modelli linguistici in tre task di tipo RAG e annota, con precisione al carattere, le porzioni di testo in conflitto con il contesto o non supportate da esso [4]. La sua distinzione tra contenuto in conflitto e contenuto privo di fondamento corrisponde alle classi *contraddetta* e *non menzionata* adottate in questa tesi.

2.5 Verifica della groundedness

Verificare una risposta rispetto alla fonte è strettamente legato alla natural language inference (NLI), il task di stabilire se una premessa implica un'ipotesi, la contraddice o è neutrale rispetto a essa. Grandi dataset di NLI come MultiNLI [33] e ANLI [34], costruito in modo avversariale, forniscono dati di addestramento, mentre VitaminC [35] aggiunge coppie contrastive in cui una revisione minima di una frase di Wikipedia ribalta il verdetto, costringendo il modello a guardare al fatto preciso e non alla sovrapposizione di parole. Applicare questi modelli a testi generati lunghi richiede di scomporli: SummaC suddivide in frasi sia il documento sia il riassunto e aggrega i punteggi di inferenza calcolati frase per frase [36], mentre FActScore scompone le generazioni lunghe in fatti atomici e li verifica uno per uno [37]. La verifica a livello di frase, adottata da GroundOne, è una via di mezzo: le frasi sono le unità naturali di una risposta, l'utente può intervenire su una frase segnalata e il numero di decisioni cresce con la lunghezza della risposta.

Nella pratica prevalgono due famiglie di verificatori. Gli encoder specializzati sono piccoli e veloci. AlignScore addestra un'unica funzione di allineamento su un'ampia miscela di task [38]; MiniCheck addestra piccoli verificatori su dati sintetici costruiti in modo da richiedere un ragionamento su più frasi, ed eguaglia modelli molto più grandi sui benchmark di grounding [39]; HHEM è un modello di valutazione delle allucinazioni dello stesso tipo, rilasciato apertamente [40]. Questi modelli valutano ogni coppia documento–frase in modo indipendente e restituiscono un'unica probabilità di supporto. La seconda famiglia usa un modello linguistico come giudice, secondo lo schema LLM-as-a-judge [41]. Framework di valutazione per RAG come RAGAS [42] e ARES [43] si basano su giudici, Lynx addestra con fine-tuning un giudice dedicato al rilevamento delle allucinazioni [44] e Self-RAG addestra il generatore stesso a emettere token di critica [45]. I giudici sono flessibili, ma il loro costo cresce con quanto scrivono, i modelli piccoli violano spesso il formato di output richiesto e per garantire una risposta interpretabile dal parser serve un decoding vincolato da una grammatica [46]. LLM-AggreFact riunisce undici dataset di grounding annotati manualmente sotto un protocollo binario comune ed è il benchmark di riferimento per entrambe le famiglie [39, 47].

2.6 Calibrazione e selective prediction

Una probabilità è calibrata quando, tra le predizioni fatte con una certa confidenza, la frazione di quelle corrette coincide con tale confidenza. Le reti neurali moderne sono spesso mal calibrate, e il temperature scaling, cioè una singola temperatura applicata ai logit e stimata su dati tenuti da parte, è una correzione semplice ed efficace [48]; l'Expected Calibration Error (ECE), calcolato su fasce di confidenza, è la misura abituale [49]. Per i modelli linguistici interrogati tramite prompt può servire anche un bias per classe che corregga una preferenza sistematica verso alcune risposte [50]. I modelli linguistici sanno, entro certi limiti, quello che sanno: sulle domande chiuse le probabilità dei loro token sono spesso informative sulla correttezza della risposta [51]. Quando le probabilità non sono accessibili, si può chiedere al modello di esprimere la confidenza a parole; per modelli grandi sottoposti a fine-tuning con feedback umano questa confidenza verbalizzata può essere calibrata meglio delle probabilità stimate per campionamento [52], ma valutazioni sistematiche la trovano grossolana e troppo sicura, con valori concentrati su pochi numeri tondi [53]. Una probabilità calibrata diventa utile grazie alla selective prediction: il sistema agisce sulle decisioni di cui è sicuro e rinvia le altre, e la curva risk–coverage mostra come il tasso di errore scenda man mano che si rinviano più decisioni [54].

2.7 Parameter-efficient fine-tuning

Specializzare un modello preaddestrato non richiede di aggiornarne tutti i pesi. La low-rank adaptation (LoRA) congela il modello e addestra, per alcune matrici di pesi, un aggiornamento a basso rango che si può fondere nei pesi o mantenere come piccolo adapter separato [55]; applicarla a tutti gli strati lineari conta più che usare un rango elevato, e si combina con pesi di base quantizzati per far stare modelli grandi in poca memoria [56]. La famiglia Qwen2.5 offre modelli aperti a partire da 0.5B parametri con un tokenizer condiviso, il che permette di confrontare dimensioni diverse con la stessa ricetta [57]. Il fine-tuning su dataset piccoli è però notoriamente instabile: i risultati variano con il seed casuale, sia attraverso l'inizializzazione dei nuovi parametri sia attraverso l'ordine dei dati di addestramento [58]; alcuni addestramenti falliscono del tutto, un comportamento legato a dettagli dell'ottimizzazione come l'assenza della bias correction in Adam [59]; e il gradient clipping, che limita la norma del gradiente, è la difesa standard contro occasionali aggiornamenti esplosivi [60].

2.8 Lacuna nella letteratura

Gli elementi necessari a un verificatore locale, veloce e calibrato esistono, ma separatamente: modelli aperti compatti e runtime che li eseguono su un laptop, l'idea System One di leggere le decisioni anziché generarle, verificatori specializzati e calibrazione post-hoc. Manca uno studio indipendente e controllato che li metta insieme su un unico task. In particolare, nessun lavoro ha misurato se un System One compatto, specializzato nella verifica della groundedness, possa eguagliare su questo task i giudici generativi e gli encoder dedicati, da dove provenga il suo vantaggio (specializzazione, readout o dimensione del modello), se le sue probabilità siano abbastanza buone da fissare soglie e rinviare decisioni, e come tempo ed energia si confrontino con quelli di un giudice sullo stesso hardware consumer. Il resto della tesi affronta queste domande.

Capitolo 3

GroundOne: metodo e implementazione

GroundOne è un modello linguistico compatto usato come System One: legge una sola volta il documento e la risposta e, senza generare testo, emette un verdetto per ciascuna frase valutata della risposta, ricavandolo dai logit di poche lettere di risposta. Il capitolo descrive la decisione che il modello prende, il prompt che raccoglie in un unico forward pass tutte le decisioni relative a una risposta, l'obiettivo di addestramento, la calibrazione delle probabilità in uscita, la regola che converte le probabilità nelle etichette restituite e il software che implementa il tutto.

3.1 Task e decisione

Per ogni frase di una risposta GroundOne sceglie fra tre classi, definite nella Sezione 4.1: rispetto al documento, la frase è *supportata*, *contraddetta* o *non menzionata*. Alle tre classi corrispondono le lettere A, B e C, e il modello restituisce una probabilità per ciascuna. Il punteggio binario di groundedness e le policy strict e lenient non sono domande distinte poste al modello, ma vengono calcolati a partire da queste tre probabilità, cosicché un unico forward pass basta per ogni utilizzo. Interrogare il modello direttamente sulle policy non porterebbe nulla: ciascuna è una funzione deterministica della classe a tre vie, per cui domande separate moltiplicherebbero le decisioni senza aggiungere informazione e potrebbero fornire al modello obiettivi di addestramento incoerenti per la stessa frase.

3.2 Tutti i verdetti in un solo forward pass

Il prompt contiene una breve istruzione, il documento e la risposta. Le frasi della risposta sono numerate, e ognuna di quelle a cui è associata una decisione è seguita da un marker, la stringa " `Verdict:`". Le frasi prive di contenuto verificabile (Sezione 4.3) restano nel testo come contesto, ma senza marker. Il modello elabora l'intera sequenza una sola volta: sull'ultimo token di ciascun marker si leggono i logit delle lettere A, B e C, normalizzati con una softmax ristretta a queste tre lettere. Poiché in un decoder ogni posizione vede soltanto quelle precedenti, il verdetto sulla frase i dipende dal documento e dalle frasi $1, \ldots, i$, ed è esattamente ciò che serve alla verifica, dato che i riferimenti, come i pronomi, puntano all'indietro. Quanto segue l'ultimo marker non influisce su alcun verdetto, quindi la sequenza si chiude lì.

Due accorgimenti rendono il readout efficiente ed esatto. Anzitutto il prompt viene costruito tokenizzando separatamente le sue parti e concatenandole, così che la posizione di ogni marker sia nota con esattezza e resti identica in addestramento e in inferenza. In secondo luogo, la proiezione di output sul vocabolario si applica solo nelle posizioni dei marker: il corpo del transformer elabora l'intera sequenza, ma la grande matrice del vocabolario viene moltiplicata soltanto per i pochi stati nascosti effettivamente letti.

Questa scelta determina il costo di una verifica. Indicando con S la lunghezza complessiva di documento e risposta, con K il numero di decisioni e con m la lunghezza in token di un marker, il System One richiede un solo prefill di $S + Km$ token. Un giudice generativo deve invece produrre anche i propri verdetti in fase di decoding: con n token per verdetto e un tempo di decoding per token t_{dec}, il suo costo è all'incirca $\text{prefill}(S) + K\,n\,t_{\text{dec}}$. Dato che m vale tre token e che sull'hardware di riferimento il decoding è, a parità di token, molto più lento del prefill, il divario si allarga al crescere di K. Sull'M4 usato in questa tesi il prefill costa circa 0.37 ms per token con il modello 0.5B e 2.2 ms con il 3B, mentre il decoding costa 5.9 e 31.8 ms per token generato: un token in decoding costa quanto quattordici-sedici token in prefill. Le due fasi, inoltre, sollecitano l'hardware in modo diverso. Il prefill elabora molti token insieme ed è limitato dalla capacità di calcolo, mentre il decoding produce un token alla volta ed è limitato dalla banda necessaria a leggere i pesi [9, 10]. La differenza riemerge nelle misure di energia del Capitolo 6.

Una seconda soluzione legge una sola volta il prefisso condiviso e vi accoda poi una breve domanda per ogni frase, valutando le domande in batch. Richiede però una KV cache replicata K volte, mentre la soluzione a marker si accontenta di un solo passaggio. È mantenuta come alternativa di riserva ed è il formato naturale per un modello non addestrato, che comprende una domanda esplicita ma non ha motivo di interpretare un marker come tale; per questo è il formato usato dalla baseline non addestrata (Sezione 5.2).

Un classificatore, non un generatore più veloce. Il readout restringe la decisione a tre lettere, e la restrizione non è un dettaglio. Dopo il marker, il modello sottoposto a fine-tuning non scriverebbe la lettera che viene letta: il suo token successivo più probabile è la stessa lettera preceduta da uno spazio, e le tre lettere del readout raccolgono insieme appena il 4–5 % circa della massa di probabilità in corrispondenza dei marker. Addestrare con una loss ristretta a queste lettere modella i loro logit relativi, non la loro probabilità assoluta, e il readout si comporta quindi come un classificatore a tre classi innestato sul modello linguistico. È questo a definire un System One: la risposta non passa mai per il decoding, ma viene scelta fra le opzioni ammissibili. Il fenomeno richiama anche l'osservazione che la probabilità del primo token di risposta e la risposta effettivamente scritta dal modello possono non coincidere [31], e che la scelta fra lettere è soggetta a bias di selezione [32]: un readout specializzato e calibrato deve assorbire entrambi gli effetti. Un esperimento di controllo che, con gli stessi pesi e lo stesso passaggio, prende al posto del readout ristretto il token che la generazione emetterebbe ne quantifica la differenza (Sezione 6.6).

3.3 Obiettivo di addestramento

GroundOne si ottiene da Qwen2.5-0.5B-Instruct e Qwen2.5-3B-Instruct [57] tramite low-rank adaptation (LoRA) [55]. Gli adapter, di rango 16, con fattore di scala 2 e dropout 0.05, sono applicati a tutti i layer lineari di tutti i blocchi transformer, poiché coprire ogni layer conta più di un rango elevato [56]. I pesi del modello base restano congelati in precisione a 8 bit. Un esempio di addestramento è una risposta con tutte le sue frasi valutate, e la loss viene calcolata su ogni marker della risposta.

La loss è una cross-entropy calcolata sulle sole tre lettere: il prompt non viene mai supervisionato e il segnale di addestramento proviene interamente dai verdetti. Poiché circa l'88 % delle frasi del training set è supportato, le classi sono pesate in modo che ciascuna contribuisca in egual misura alla loss: con la distribuzione del training set si ottengono pesi di circa 0.38 per le frasi supportate, 7.5 per quelle contraddette e 4.4 per quelle non menzionate. Senza questi pesi un modello può ottenere una loss bassa rispondendo quasi sempre "supportata". Ne discende una conseguenza utile: un modello che avesse appreso soltanto le frequenze delle classi resterebbe a una loss di $\ln 3 \approx 1.10$, quindi una loss inferiore a questo valore indica che il testo viene effettivamente sfruttato.

L'ottimizzazione usa AdamW senza weight decay e, come nella configurazione predefinita di MLX, senza bias correction delle stime dei momenti; non si applica gradient clipping. Il learning rate di 10^{-4} viene raggiunto con un warm-up lineare sul primo 3 % dei passi e poi portato a zero con un cosine schedule, nell'arco

di un'epoca sulle risposte del training set. Le risposte di lunghezza simile sono raggruppate in micro-batch entro un budget di token comprensivo del padding, e i gradienti vengono accumulati in modo che ogni passo dell'ottimizzatore veda circa 16 risposte, ovvero circa 110 decisioni: così ogni passo contiene qualche frase contraddetta nonostante la rarità della classe. Il gradient checkpointing mantiene l'occupazione di memoria entro i 16 GB del portatile. Il modello viene valutato periodicamente sul dev set e si conserva il checkpoint con la balanced accuracy a tre classi più alta, mediata sui tre task. La Sezione 5.1 riporta le impostazioni di ciascun run.

3.4 Calibrazione

I pesi delle classi addestrano il modello come se le tre classi fossero ugualmente frequenti, per cui le sue probabilità grezze non rispecchiano la frequenza reale degli errori. Sul dev set si stima quindi una trasformazione post-hoc: i tre logit vengono divisi per una temperatura T e traslati di un bias per classe, $\mathrm{softmax}(z/T + b)$, con il bias della classe supportata fissato a zero, e T e b sono scelti minimizzando la negative log-likelihood delle etichette del dev set [48]. Il bias svolge il ruolo della correzione contestuale proposta per i modelli linguistici interrogati tramite prompt [50], qui però stimata su dati etichettati. La sola temperatura cambia le probabilità ma non la classe più probabile né l'ordinamento delle frasi. Il bias, invece, può cambiare la classe più probabile e, con tre classi, può anche riordinare leggermente $P(\text{supported})$ fra una frase e l'altra. Per questo le metriche di classificazione e l'AUROC sono calcolate sull'output grezzo, mentre le probabilità calibrate si usano dove una probabilità deve corrispondere a una frequenza: nelle metriche di calibrazione, nelle soglie e nella selective prediction (Sezione 6.7).

La calibrazione è misurata principalmente con l'Expected Calibration Error (ECE) [49, 48]. Le predizioni vengono ripartite in base alla confidenza in M intervalli di uguale ampiezza, con $M = 15$ in questa tesi. Per ogni intervallo B_m si confronta la confidenza media $\mathrm{conf}(B_m)$ con la frequenza osservata $\mathrm{acc}(B_m)$, e gli scarti vengono mediati con pesi proporzionali alla numerosità degli intervalli:

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{n} \Big|\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\Big|. \tag{3.1}$$

Nella variante binaria la confidenza è $P(\text{supported})$ e la frequenza osservata è la quota di frasi supportate nell'intervallo; nella variante top-label a tre classi la confidenza è la probabilità massima e la frequenza osservata è la quota di predizioni corrette. L'ECE non confronta la distribuzione delle classi predette con quella del test set: verifica se, fra le frasi a cui il modello assegna una certa confidenza, quella confidenza corrisponde alla realtà. Dipende dalla scelta degli intervalli, e

un predittore costante che restituisce sempre la frequenza di base ottiene un ECE vicino a zero pur non discriminando nulla. Per questi motivi l'ECE è sempre riportato insieme al Brier score, alla negative log-likelihood e ai reliability diagram, mentre la capacità delle probabilità di separare le decisioni corrette da quelle errate si misura a parte, con l'AUROC e con l'area sotto la risk–coverage curve.

Dalle probabilità alle etichette restituite. Chi usa il verificatore vede per ogni frase un'etichetta e una probabilità, che devono essere coerenti tra loro. La decisione grezza, cioè la classe più probabile secondo l'output non calibrato, è quella valutata in questa tesi, ma può contraddire la probabilità calibrata mostrata accanto: una frase può essere etichettata "non menzionata" pur avendo una $P(\text{supported})$ calibrata di poco superiore a un mezzo. L'output restituito ricava quindi ogni elemento dalle probabilità calibrate con un'unica regola. Una frase è etichettata come supportata se e solo se $P(\text{supported}) \geq \tau$; altrimenti riceve la più probabile fra le due classi restanti. La policy strict segnala ogni frase non supportata, quella lenient soltanto le contraddette. La soglia τ è stimata sul dev set in modo da massimizzare la balanced accuracy a tre classi, e l'utente può modificarla a ogni richiesta per scegliere un diverso punto operativo senza riaddestrare il modello.

3.5 Software

GroundOne è implementato come pacchetto Python installabile basato su MLX [3, 14], il framework per array pensato per Apple silicon, e tutte le fasi di addestramento e di inferenza descritte finora vengono eseguite al suo interno sullo stesso portatile. Il pacchetto comprende il costruttore del dataset con i relativi controlli, il ciclo di addestramento, la calibrazione, la valutazione sui test set congelati, le baseline e un meccanismo di protezione della memoria che esegue i processi lunghi uno alla volta e interrompe un processo prima che il sistema esaurisca la memoria. Il verificatore è esposto tramite un'interfaccia a riga di comando e un server HTTP; il server offre un endpoint nativo e un endpoint compatibile con l'interfaccia usata da altri modelli System One, così che i client esistenti possano adottare GroundOne cambiando soltanto l'indirizzo. L'endpoint nativo riceve un documento e una risposta, segmenta la risposta con la stessa regola usata per costruire i dati, esegue il singolo forward pass della Sezione 3.2 e riporta per ogni frase se è stata valutata, le tre probabilità calibrate, l'etichetta restituita, i flag delle policy strict e lenient, nonché la regola di decisione e la soglia che li hanno prodotti; indica inoltre il numero di token in ingresso, il numero di token generati, sempre pari a zero, e il tempo impiegato. L'endpoint compatibile risponde a domande chiuse generiche su un contesto condiviso, con il formato a una domanda per frase. Tutto il calcolo del modello avviene su un unico worker thread di lunga durata, perché il contesto Metal

della GPU non deve spostarsi da un thread all'altro, e nulla esce dalla macchina: modelli, dati e richieste restano in locale.

Capitolo 4

Dati

GroundOne apprende un'unica decisione, e i dati devono insegnare esattamente quella decisione e misurarla in modo affidabile. Il capitolo definisce anzitutto la decisione e le sue etichette, poi descrive i tre dataset pubblici da cui esse provengono e il modo in cui vengono trasformati in training set, development set e test set. Le ultime sezioni mostrano come l'affidabilità dei test set sia stata verificata senza nuove annotazioni, che cosa emerge da un'analisi esplorativa del risultato e come sia stato escluso il leakage tra dati di addestramento e dati di test.

4.1 Task ed etichette

L'input è costituito da un documento recuperato e da una risposta scritta da un modello linguistico, suddivisa in frasi. Per ogni frase il modello stabilisce se, *rispetto al documento*, essa è *supportata*, *contraddetta* o *non menzionata*. Una frase è supportata quando tutto ciò che afferma è detto o direttamente implicato dal documento; è contraddetta quando una sua parte è in conflitto con il documento; è non menzionata quando aggiunge informazioni che il documento non contiene né contraddice. Una frase che contiene sia una parte in conflitto sia una parte non verificabile è etichettata come contraddetta, perché la contraddizione è l'errore più grave. Conta solo il documento: un'affermazione che risulti vera nel mondo reale ma che sia assente dal documento resta non menzionata.

Il numero di decisioni per una risposta coincide con il numero delle sue frasi valutate, indicato con K. Questa grandezza è centrale nella tesi. Un System One legge documento e risposta una sola volta, per cui ogni decisione aggiuntiva è quasi gratuita, mentre un giudice generativo deve produrre testo per ogni frase. Il confronto con i giudici generativi è quindi condotto in funzione di K, e i dati devono comprendere risposte lunghe oltre che brevi.

Il modello restituisce tre probabilità per frase, e ogni altra grandezza è derivata da queste anziché appresa separatamente. Il punteggio binario di groundedness, usato per il confronto con i benchmark binari pubblicati, è $P(\text{supported})$. Una policy *strict*, per cui è un errore tutto ciò che il documento non afferma, segnala una frase quando $P(\text{supported})$ scende sotto una soglia τ. Una policy *lenient*, per cui sono errori solo le contraddizioni, la segnala soltanto quando è giudicata contraddetta. L'utente può così scegliere in fase di inferenza la nozione di errore e la soglia, senza riaddestrare il modello; la Sezione 3.4 riporta la regola esatta con cui il verificatore produce le etichette restituite.

4.2 Dataset di partenza

Il dataset è costruito di proposito a partire dal minor numero possibile di fonti: una per l'addestramento e tre per il test (Tabella 4.1). Meno fonti significano un'unica definizione delle etichette, un unico formato del testo e meno punti in cui la costruzione può introdurre errori.

Tabella 4.1: Dataset di partenza e loro ruolo in GroundOne.

Dataset	Licenza	Ruolo	Etichette
RAGTruth [4]	MIT	training set, dev set, test T1	span di caratteri relativi a quattro tipi di allucinazione nelle risposte di LLM
LLM-AggreFact [39, 47]	CC BY-ND 4.0	solo test T2	binarie (supportata / non supportata) per affermazione
VitaminC [35]	CC BY-SA 3.0	solo test T3	supports / refutes / not enough information

RAGTruth. RAGTruth [4] contiene 17,790 risposte scritte da sei modelli linguistici (GPT-3.5, GPT-4, Llama-2 7B, 13B e 70B, e Mistral-7B) in tre task di retrieval-augmented generation (RAG). Nel question answering il contesto è un insieme di passaggi recuperati per una domanda di MS MARCO [61]; nella summarization è un articolo di cronaca tratto da CNN/DailyMail [62, 63] o da notizie recenti; nella generazione data-to-text è la scheda di un'attività commerciale di Yelp in formato JSON [64]. Due annotatori formati hanno individuato, in ciascuna risposta, gli span di caratteri non supportati dal contesto, con una terza revisione in caso di disaccordo, e hanno assegnato a ogni span uno di quattro tipi: *conflitto* evidente o sottile e *informazione infondata* evidente o sottile. L'articolo riporta

un accordo tra annotatori del 91.8 % a livello di risposta e del 78.8 % a livello di span. È l'unico dataset pubblico che unisca risposte naturali di più frasi, prodotte da modelli diversi, alla distinzione tra contenuto contraddetto e contenuto non supportato, ed è per questo l'unica fonte di addestramento. Lo split ufficiale separa 450 fonti di test dalle restanti fonti, destinate all'addestramento.

LLM-AggreFact. LLM-AggreFact [39, 47] aggrega undici dataset di fact checking annotati manualmente, composti da coppie (documento, affermazione). Coprono la summarization (AggreFact-CNN e XSum [65], TofuEval [66]), il question answering basato su retrieval e in forma estesa (RAGTruth [4], ClaimVerify [67], ExpertQA [68], LFQA [69]) e il fact-checking vero e proprio (WiCE [70], REVEAL [71], FactCheck-GPT [72]). La classifica pubblica associata rende i risultati confrontabili con quelli dei sistemi pubblicati. La licenza vieta di usare i dati per addestrare modelli [73]; in questa tesi il dataset serve quindi soltanto alla valutazione, e nessun addestramento, calibrazione o selezione avviene su alcuno dei suoi split.

VitaminC. VitaminC [35, 74] è un dataset di verifica dei fatti costruito a partire da revisioni di Wikipedia che cambiano un fatto. Per ogni revisione, annotatori formati hanno scritto affermazioni supportate da una versione della frase e confutate dall'altra; quando non era possibile formulare una contraddizione, la coppia è stata etichettata come priva di informazioni sufficienti. Poiché le due versioni dell'evidenza sono quasi identiche, il modello deve guardare al fatto preciso e non alla sovrapposizione lessicale. Su un campione ri-annotato di 2,000 coppie gli autori riportano un κ di Fleiss pari a 0.71. In questa tesi VitaminC ha un ruolo preciso. RAGTruth non pubblica l'accordo tra annotatori sul *tipo* di allucinazione, e da solo non può quindi dimostrare che il confine tra contraddetta e non menzionata venga appreso. VitaminC fornisce proprio questo confine, con etichette di affidabilità nota e in un dominio diverso.

Diversi altri candidati sono stati esaminati e scartati. I corpora di natural language inference come ANLI [34] hanno premesse brevi, lontane dai contesti tipici della RAG, e VitaminC copre la stessa distinzione a tre classi su testo fattuale. I dati di addestramento sintetici o solo binari, come quelli rilasciati con MiniCheck [39], non hanno etichette a tre classi. Sono stati infine esclusi i dataset con etichette generate da un modello linguistico o che riutilizzano risposte di test di RAGTruth, per mantenere i test set indipendenti e le etichette umane.

4.3 Costruzione

Segmentazione in frasi. Le risposte vengono prima spezzate in corrispondenza degli a capo, perché contengono spesso elenchi e intestazioni, e ogni riga viene poi

segmentata con PySBD [75], un segmentatore a regole che gestisce abbreviazioni e numeri decimali. I simboli di elenco iniziali come "3." o "*" vengono rimossi, perché riguardano l'impaginazione e non il contenuto. Lo stesso segmentatore è usato in addestramento, valutazione e inferenza, così che il modello veda sempre frasi tagliate allo stesso modo.

Frasi senza contenuto verificabile. Alcune frasi non affermano nulla sul documento: saluti, intestazioni che introducono un elenco, righe con la sola citazione come "(Passage 2)" e formule di rifiuto. Restano nella risposta numerata come contesto, ma non sono decisioni e non hanno etichetta. Una breve regola le individua e si applica all'8.2 % di tutte le frasi di RAGTruth, in gran parte intestazioni di elenchi. La regola riguarda solo la formattazione (simboli di elenco, righe di sola citazione e formule conversazionali) e non modifica mai un'etichetta; la sua copertura è stata verificata rispetto alla segmentazione indipendente di LLM-AggreFact descritta nella Sezione 4.5.

Etichette. Una frase valutata è etichettata come contraddetta se si sovrappone a uno span di conflitto, come non menzionata se si sovrappone solo a span di informazione infondata, e come supportata negli altri casi. Solo 27 dei 14,226 span annotati (0.19 %) non toccano alcuna frase valutata, per cui la conversione non perde quasi nessuna annotazione. Per LLM-AggreFact l'etichetta binaria è mantenuta così come rilasciata. Le etichette di VitaminC sono mappate direttamente: *supports* in supportata, *refutes* in contraddetta e *not enough information* in non menzionata.

Esclusioni e split. Vengono rimosse le risposte che RAGTruth segnala come rifiuti scorretti o come troncate, le risposte senza alcuna frase valutata e quelle in cui documento e risposta superano insieme i 2,048 token, limite di lunghezza imposto dalla memoria del portatile da 16 GB. Lo split di test ufficiale di RAGTruth diventa il test set T1. Le fonti di addestramento vengono mescolate con un seed fisso: il 10 % di esse forma il dev set e il resto il training set, così che risposte relative alla stessa fonte non compaiano mai in entrambi. Vengono inoltre rimosse sei risposte di addestramento relative a un articolo di cronaca presente anche nel test set di LLM-AggreFact. Dalla parte reale dello split di test di VitaminC, che è sbilanciata, si campionano con un seed fisso 1,000 coppie per classe, che formano T3.

4.4 Il dataset risultante

La Tabella 4.2 riassume i cinque file. Il training set contiene 13,203 risposte e 90,795 frasi etichettate. I record sono memorizzati per risposta, con un'etichetta

per frase, così che gli stessi dati possano alimentare modi diversi di leggere le K decisioni di una risposta.

Tabella 4.2: Il dataset GroundOne. Una decisione è una frase etichettata; T2 è binario.

Split	Fonte	Record	Decisioni	Supportate	Contraddette	Non menzionate
Train	RAGTruth	13,203	90,795	79,898	4,039	6,858
Dev	RAGTruth	1,468	9,996	8,827	410	759
T1	RAGTruth test	2,647	17,284	15,745	584	955
T2	LLM-AggreFact test	29,320	29,320	22,832	6,488 non supportate	
T3	VitaminC test	3,000	3,000	1,000	1,000	1,000

RAGTruth è fortemente sbilanciato: circa l'88 % delle frasi etichettate è supportato e le contraddizioni sono appena dal 3 al 4 % (Figura 4.1). È la frequenza naturale degli errori nelle risposte dei modelli, e viene preservata: il test set riflette ciò che un verificatore incontra nella pratica, mentre lo sbilanciamento è gestito in fase di addestramento. T3 compensa con un test bilanciato a tre classi. Le risposte hanno in media 6.9 frasi valutate nel training set e 6.5 in T1, con un massimo di 28 (Figura 4.2). Per l'analisi in funzione della lunghezza, T1 è diviso in quattro fasce di K: 756 risposte con 1–4 frasi, 1,288 con 5–8, 497 con 9–13 e 106 con 14 o più. Le celle con meno di 30 frasi di una classe minoritaria, come le frasi contraddette nel question answering, sono riportate ma segnalate come indicative.

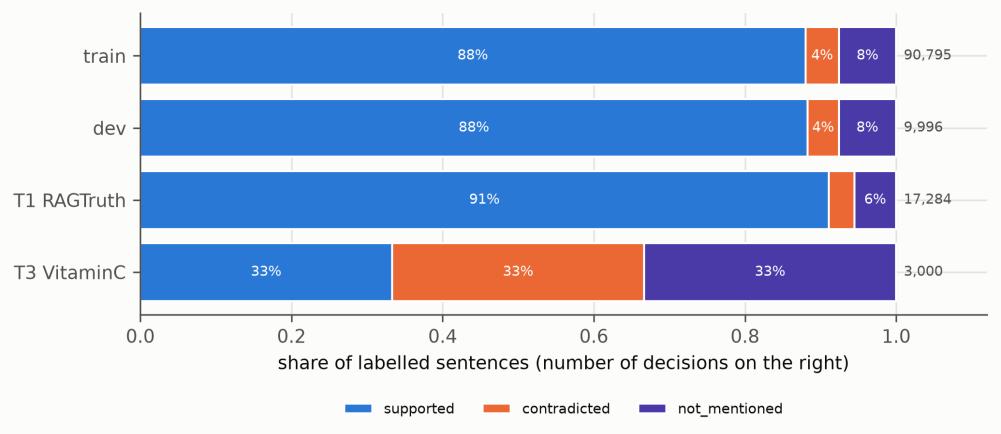


Figura 4.1: Distribuzione delle classi delle frasi etichettate in ciascuno split. RAGTruth (train, dev e T1) è supportato per circa l'88–91 %; T3 è campionato a 1,000 coppie per classe.

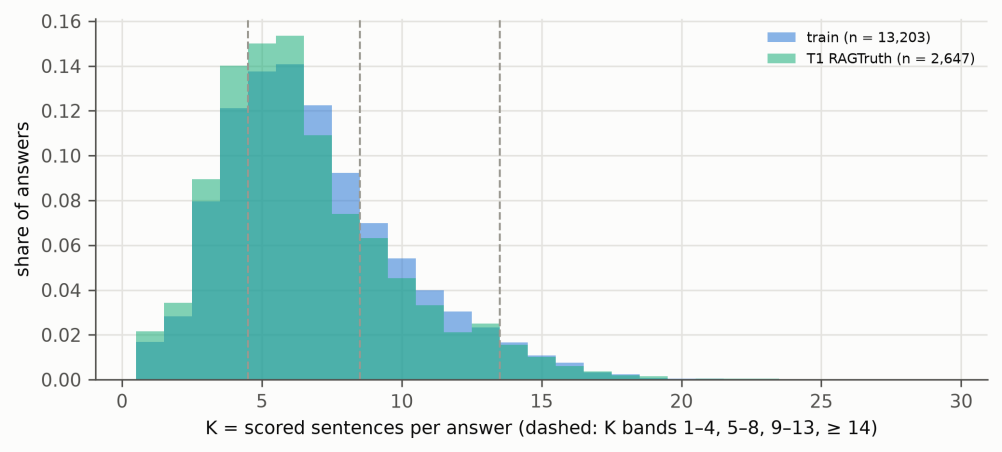


Figura 4.2: Distribuzione di K, il numero di frasi valutate per risposta, nel training set e in T1. Le linee tratteggiate indicano le fasce usate per riportare i risultati in funzione di K.

4.5 Affidabilità dei test set

I test set sono stati verificati senza nuove annotazioni, ricorrendo a evidenze indipendenti dalla costruzione descritta sopra (Tabella 4.3). Tutte le etichette di test provengono dagli annotatori umani dei dataset pubblicati. Le due scelte che potevano introdurre errori, cioè la segmentazione e la conversione dagli span alle etichette di frase, sono state confrontate con LLM-AggreFact, i cui autori hanno convertito in modo indipendente le stesse risposte di test di RAGTruth in affermazioni a livello di frase. Il segmentatore ritrova esattamente il 96.3 % delle loro affermazioni, e solo l'8.5 % delle frasi valutate manca dal loro insieme: si tratta per lo più di frasi eliminate dal loro filtro, non documentato, delle affermazioni "non meritevoli di verifica". Sulle 15,492 frasi presenti in entrambi, le etichette binarie concordano nel 98.75 % dei casi, e tutti i 193 disaccordi vanno nella stessa direzione: la regola basata sugli span marca come non supportata una frase che LLM-AggreFact considera supportata. Infine, una baseline basata su encoder, valutata su T2 senza modifiche, riproduce il proprio punteggio pubblicato (74.0 contro 73.5), il che convalida il codice di valutazione.

Questi controlli convalidano il versante binario di T1. La distinzione tra contraddetta e non menzionata poggia invece sugli annotatori di RAGTruth, il cui accordo sul tipo di allucinazione non è pubblicato. Inoltre, una recente ri-annotazione dello split di test di RAGTruth ha trovato 1.68 volte più risposte con allucinazioni rispetto alle etichette originali [76], il che suggerisce che T1 sottostimi le due classi minoritarie. Per questi motivi i risultati a tre classi su T1 vengono sempre letti insieme a quelli su T3. Dopo questi controlli, e prima di addestrare qualsiasi

Tabella 4.3: Controlli di affidabilità eseguiti prima del congelamento dei test set.

Controllo	Criterio	Esito
Provenienza delle etichette	ogni etichetta di test proviene da annotatori umani	superato
Segmentazione	$\geq$ 90 % delle affermazioni di LLM-AggreFact recuperate, $\leq$ 10 % delle frasi assenti da esso	96.3 % / 8.5 %
Derivazione delle etichette	accordo $\geq$ 97 % con le etichette binarie di LLM-AggreFact	98.75 %
Baseline di riferimento	MiniCheck-RoBERTa su T2 entro 1.5 punti dal punteggio pubblicato	74.0 contro 73.5
Leakage	nessuna fonte, documento o coppia (documento, frase) in comune tra addestramento e test	superato

modello, i tre file di test sono stati congelati con hash crittografici, e il costruttore del dataset si rifiuta di sovrascriverli con contenuti diversi; ricostruendo il dataset dalle fonti grezze si riottengono esattamente i file congelati.

4.6 Analisi esplorativa

Un'analisi esplorativa delle fonti e del dataset costruito ha messo in luce quattro proprietà che condizionano la lettura dei risultati.

Gli errori dipendono dal task e dal modello. Le risposte data-to-text contengono allucinazioni con una frequenza da due a quattro volte maggiore rispetto a quelle degli altri task; le risposte di Llama-2 e Mistral concentrano la maggior parte delle allucinazioni, mentre quelle di GPT-3.5 e GPT-4 sono quasi sempre fedeli al documento (Figura 4.3). I risultati sono quindi riportati per task, poiché una media unica rifletterebbe soprattutto il data-to-text.

Le risposte di test sono più pulite di quelle di addestramento. In ogni task e per ogni generatore, T1 ha una quota di frasi contraddette e non menzionate più bassa del training set; nel question answering, per esempio, è contraddetto lo 0.6 % delle frasi di T1 contro l'1.6 % del training set. Le metriche che dipendono dalla frequenza di base, come la precisione o l'F1 sulle classi minoritarie, non sono quindi confrontabili tra dev set e test set. Le metriche principali sono la balanced accuracy, il recall macro e l'AUROC, che non ne dipendono.

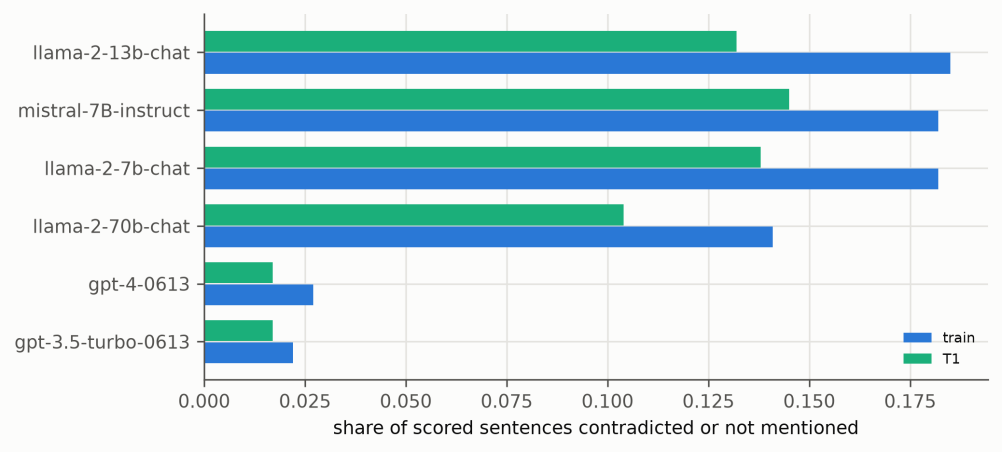


Figura 4.3: Quota di frasi contraddette o non menzionate in base al modello che ha scritto la risposta, nel training set e in T1.

Le contraddizioni sono per lo più evidenti. Circa l'85 % delle frasi contraddette del training set deriva da span annotati come conflitti *evidenti*, come numeri o nomi sbagliati, e solo 147 da conflitti *sottili*. La classe contraddetta è quindi composta soprattutto da contraddizioni palesi, e le contraddizioni sottili non possono essere valutate separatamente.

Esistono indizi superficiali, e definiscono le baseline. Due indizi semplici predicono le etichette senza verificare la frase rispetto al documento. Il primo è la sovrapposizione lessicale: la quota di parole di contenuto di una frase che compaiono anche nel documento è più alta per le frasi supportate che per quelle contraddette e non menzionate, in tutti e tre i test set (Figura 4.4). Da sola, separa le frasi supportate da quelle non supportate con un AUROC di 0.74 su T1 e di 0.70 su T3. Il secondo indizio è la formulazione della frase. Un classificatore che vede soltanto la frase raggiunge una balanced accuracy a tre classi di 0.66 nel data-to-text, perché alcuni dettagli, come il numero di recensioni o la disponibilità di Wi-Fi o del parcheggio con valet, vengono spesso inventati nelle descrizioni di Yelp. Nel question answering e nella summarization lo stesso classificatore resta al livello del caso (0.36 e 0.35), e così pure su T3 (0.33), dove la costruzione contrastiva del dataset elimina l'indizio. Un verificatore è utile solo nella misura in cui batte questi indizi, per cui entrambi sono riportati come baseline, task per task.

Infine, le frasi non supportate non si concentrano alla fine della risposta. La prima frase è raramente non supportata; la quota raggiunge il massimo tra la terza e la quinta posizione e resta tra l'8 e l'11 % nelle posizioni successive (Figura 4.5). Anche le risposte lunghe contengono errori verso la fine, per cui l'accuratezza per valori elevati di K è rilevante.

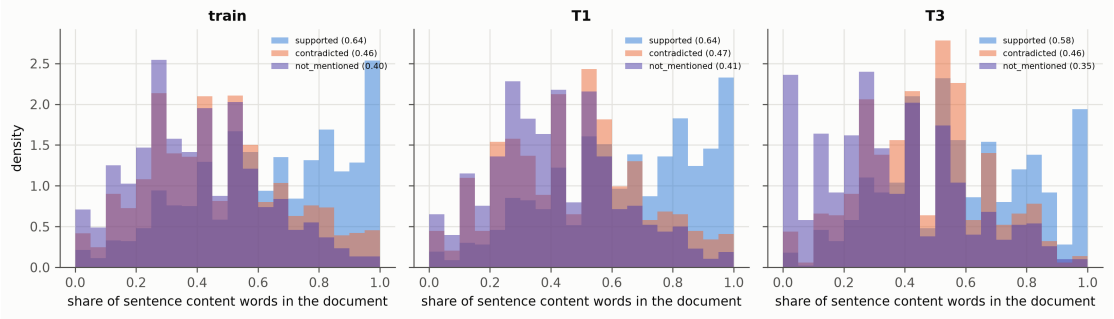


Figura 4.4: Sovrapposizione lessicale tra una frase e il suo documento, per classe, nel training set, in T1 e in T3 (media tra parentesi).

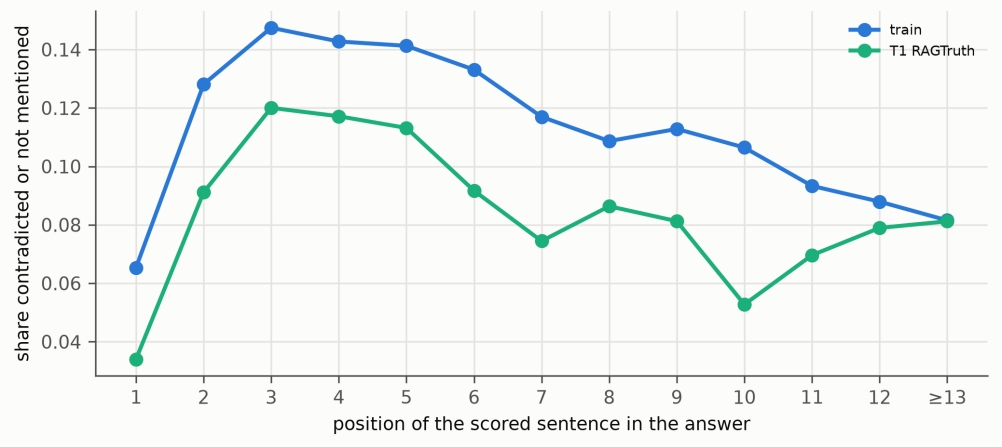


Figura 4.5: Quota di frasi contraddette o non menzionate in base alla posizione nella risposta.

4.7 Controllo del leakage

Gli split sono definiti per fonte, mai per frase, e a ogni costruzione del dataset viene eseguito un controllo automatico. Il controllo fallisce se una fonte, un documento o una coppia (documento, frase) del training set o del dev set compare in uno qualsiasi dei test set, oppure se dev set e training set condividono una fonte. Il controllo risulta superato. Segnala inoltre le stringhe di frase che compaiono sia nei dati di addestramento sia in quelli di test con documenti diversi: sono 788 frasi template, come “The business has a rating of 4.0 stars.”, attese nel data-to-text e che non rivelano alcuna etichetta di test. Non è possibile verificare se i dataset pubblici usati qui facessero parte dei dati di pre-training di Qwen2.5, e questo resta un limite comune alla maggior parte dei lavori su benchmark pubblici.

Capitolo 5

Impostazione sperimentale

Questo capitolo presenta i modelli addestrati, i sistemi con cui sono messi a confronto, le metriche, il protocollo statistico e le modalità di misura di tempo ed energia. Tutti gli esperimenti si svolgono su un'unica macchina, un MacBook Pro con chip Apple M4 (CPU a 10 core, GPU a 10 core, banda di memoria di 120 GB/s) e 16 GB di memoria unificata [13]: ogni risultato riflette così ciò che un verificatore può fare sullo stesso dispositivo su cui gira un assistente locale. Anche tutte le baseline, compresi gli encoder e il System One generalista, sono eseguite su questo portatile, e nessun risultato dipende da un servizio remoto.

5.1 Modelli

Entrambi i modelli GroundOne partono da Qwen2.5 [57] nella versione instruction-tuned, convertita in MLX con pesi a 8 bit, e sono addestrati con la procedura descritta nella Sezione 3.3. Si distinguono soltanto per la dimensione, per il numero di risposte di addestramento e per la composizione dei micro-batch, che dipende dalla memoria disponibile.

Il modello 0.5B è addestrato sull'intero training set di 13,203 risposte. Un'epoca di 802 passi di ottimizzazione ha richiesto 5.5 ore di tempo wall-clock, con una valutazione sull'intero dev set ogni 150 passi, e un'occupazione massima di 2.6 GB di memoria. Il modello 3B è circa sei volte più lento per token; per questo viene addestrato per una notte su un sottoinsieme casuale di 6,500 risposte (circa 45,000 decisioni), con 403 passi in 12.25 ore e un picco di 5.9 GB. La sua valutazione sul dev set, ogni 80 passi, usa un sottoinsieme fisso di 200 risposte, dato che una valutazione completa richiederebbe circa un'ora. In entrambi i casi si conserva il checkpoint con il miglior punteggio sul dev set, che viene poi calibrato come descritto nella Sezione 3.4; per il 3B si tratta del passo 320.

Per stimare quanto i risultati dipendano dal seed, che determina l'inizializzazione degli adapter e l'ordine dei batch, la procedura del 0.5B viene ripetuta con un secondo seed: è noto che l'esito del fine-tuning varia con entrambi questi fattori [58]. Un'ulteriore esecuzione con un terzo seed è collassata presto su una predizione costante ed è stata interrotta a circa il 60 % dell'epoca; poiché incompleta, è esclusa dalle tabelle e viene discussa nella Sezione 6.2. La seconda esecuzione, portata a termine, è valutata su tutti e tre i test set esattamente come la prima.

5.2 Baseline

Le baseline servono a isolare l'origine di un eventuale vantaggio, e ciascuna risponde a una domanda precisa.

Segnali che non leggono mai il documento nella sua interezza. Due predittori elementari fissano la soglia minima che qualsiasi verificatore deve superare. Il primo assegna a una frase un punteggio pari alla frazione delle sue parole di contenuto che compaiono anche nel documento. Il secondo è un classificatore bag-of-words addestrato sulle sole frasi del training set, senza il documento. Entrambi sono descritti nella Sezione 4.6.

Readout senza addestramento (B1). Agli stessi modelli Qwen2.5, senza fine-tuning, si pone per ogni frase la domanda a tre classi e se ne leggono i logit delle lettere di risposta. Un modello non addestrato non ha alcun motivo di interpretare il marker `Verdict:` come una domanda, perciò questa baseline adotta il formato con una domanda esplicita per frase (Sezione 3.2). Misura quindi ciò che il meccanismo System One offre prima di qualunque specializzazione.

Giudici generativi (B0a, B0b). I modelli non addestrati sono impiegati anche nel modo più diffuso nella pratica: ricevono il documento, la risposta numerata e l'elenco delle frasi da verificare, e scrivono un verdetto per ogni frase con greedy decoding. Il giudice *sintetico* produce una riga "`<number>: <letter>`" per frase; il giudice *verboso* scrive una breve frase di ragionamento seguita dal verdetto. Un verdetto che non è possibile interpretare costituisce un errore di formato: viene contato come errore nel calcolo dell'accuratezza e riportato a parte.

Giudici a parità di output (B0d). I giudici sintetico e verboso restituiscono meno di GroundOne, cioè una lettera per frase e nessuna probabilità, e non consentono quindi né calibrazione, né una soglia regolabile, né la scelta di una politica di decisione. Per questo a due ulteriori giudici si chiede esattamente ciò che produce GroundOne: un'etichetta e tre probabilità per frase. Il primo le scrive in un

oggetto JSON, con le probabilità espresse a parole, che è quanto uno sviluppatore otterrebbe con il solo prompt. Il secondo è il giudice sintetico, con le tre probabilità lette dai logit in corrispondenza della lettera che genera; ottiene lo stesso contenuto al costo del giudice sintetico ed è l'avversario generativo più forte. Entrambi sono calibrati con la stessa temperatura e lo stesso bias di GroundOne, stimati su 200 risposte del dev set, così che le loro probabilità ricevano lo stesso trattamento. Il prompt JSON è stato scelto su dodici risposte del dev set badando unicamente alla validità del formato: i modelli copiavano alla lettera un esempio numerico presente nel prompt, che è stato perciò sostituito da segnaposto. Il parsing è permissivo, a vantaggio dei giudici: le voci complete di un oggetto troncato vengono conservate e un'etichetta mancante si ricava dalle probabilità. Il decoding non è vincolato da una grammatica [46], in modo che il protocollo resti quello degli altri giudici; questa scelta pesa soprattutto sugli errori di formato del modello più piccolo.

Readout e generazione a confronto (B2). I modelli GroundOne addestrati sono eseguiti con lo stesso prompt e lo stesso singolo forward pass, ma il verdetto a ogni marker è il token con la probabilità più alta sull'intero vocabolario, cioè quello che produrrebbe la generazione greedy di un solo token. Le tre lettere, con o senza spazio iniziale, corrispondono alle tre classi, e qualunque altro token è un errore di formato. Questo controllo lascia invariati pesi, prompt e costo, e cambia soltanto se la decisione sia ristretta o meno alle opzioni ammissibili.

Un System One generalista (B3). Il progetto open Rizzo-Flow [8] realizza l'idea di System One con un modello da 4B sottoposto a fine-tuning su decisioni tipizzate generiche. Gira in un server dedicato, basato su llama.cpp [2], e viene interrogato via HTTP passando il documento e la risposta numerata come stato condiviso e una domanda a scelta fra tre opzioni per ogni frase. Si usa il modello quantizzato a 4 bit perché quello a 8 bit non entra in memoria insieme al sistema operativo.

Un decision model generalista della stessa dimensione (B3b). d1-3B di Liquid AI [26] è un decision model open da 3.1B parametri, post-addestrato per decisioni tipizzate su un'ampia varietà di task, che legge lettere di risposta ristrette con lo stato condiviso tra le domande: appartiene alla stessa famiglia di readout di GroundOne e ha la dimensione del suo modello più grande, ma è generalista. Consente così di isolare il valore della specializzazione rispetto a un decision model già addestrato a decidere, anziché rispetto a un modello non addestrato. Viene eseguito zero-shot con il proprio codice di inferenza, in PyTorch sulla GPU del portatile e senza accesso alla rete, con lo stesso stato e le stesse domande di Rizzo-Flow e al più sei domande per chiamata sullo stesso stato; su LLM-AggreFact i documenti sono suddivisi esattamente come per GroundOne e si

conserva il massimo sui frammenti. Ai fini della riproducibilità conta un dettaglio pratico: sulla GPU Apple PyTorch mantiene i kernel compilati per ogni nuova lunghezza di input senza mai rilasciarne la memoria, perciò la valutazione si svolge in processi worker che vengono riavviati quando la memoria occupata dal driver della GPU supera 11 GB, senza alcun effetto sugli output.

Encoder specializzati (B4). MiniCheck-RoBERTa [39] e HHEM-2.1-open [40] sono i verificatori di piccole dimensioni più forti oggi disponibili. Valutano ogni coppia (documento, frase) in modo indipendente e restituiscono un'unica probabilità di supporto; i documenti lunghi vengono suddivisi in frammenti di circa 400 parole e si conserva il massimo sui frammenti, come in MiniCheck. Poiché non hanno un output a tre classi, sono confrontati soltanto sulla decisione binaria. Girano in PyTorch sulla GPU del portatile.

Risultati pubblicati. Per LLM-AggreFact, i punteggi di sistemi più grandi presenti nella leaderboard pubblica, come Qwen2.5-7B usato zero-shot, sono riportati così come pubblicati, senza rieseguirli.

I sistemi più lenti sono valutati su sottocampioni stratificati dei test set: RizzoFlow su 300 risposte di T1, 600 coppie di T3 e 1,100 elementi di T2, di cui ne ha valutati 1,093 (sette eccedevano il suo contesto); d1-3B su un sottocampione casuale di 4,000 elementi di T2; il 3B non addestrato su 600 risposte di T1; i giudici generativi sulle 200 risposte del protocollo di latenza descritto più avanti. GroundOne, il 0.5B non addestrato, gli encoder e, su T1 e T3, d1-3B sono valutati sui test set completi, e ogni confronto con un sistema sottocampionato è condotto sugli elementi valutati da entrambi i sistemi.

5.3 Metriche

Poiché la quota di frasi allucinate non è la stessa nel dev set e in T1 (Sezione 4.6), le metriche primarie sono scelte in modo da non dipendere dal tasso di base. Per la decisione a tre classi la metrica principale è la balanced accuracy, cioè la recall media sulle tre classi, affiancata dalla recall di ciascuna classe e dalla frazione di predizioni per classe, che rivela un modello collassato su un'unica risposta. Per la decisione binaria si riportano la balanced accuracy con soglia 0.5, come nella leaderboard di LLM-AggreFact, e l'AUROC di $P(\text{supported})$, che non dipende da alcuna soglia. La calibrazione è misurata con l'ECE definito nella Sezione 3.4, con il Brier score e con la negative log-likelihood; la selective prediction con le curve risk–coverage e l'area sottesa (AURC) [54]. Tutte le metriche su T1 sono riportate anche per task e per fascia di K.

5.4 Protocollo statistico

Ogni differenza tra due sistemi è accompagnata da un intervallo di confidenza al 95 % ottenuto con un paired bootstrap con 2,000 resampling [77, 78, 79]. L'unità di resampling non è la frase, ma il gruppo impiegato nel controllo di leakage: il documento sorgente in RAGTruth, il documento in LLM-AggreFact e la pagina di Wikipedia in VitaminC. Le frasi di una stessa risposta, e le risposte relative a uno stesso documento, sono fortemente correlate, e ricampionarle in modo indipendente porterebbe a intervalli troppo stretti. Su LLM-AggreFact i gruppi sono ricampionati all'interno di ciascuno degli undici dataset, perché la media macro assegna a ogni dataset lo stesso peso. I due sistemi di un confronto sono valutati sugli stessi resampling, e il p-value riportato è la frazione bilaterale di resampling in cui la differenza cambia segno.

Due limiti del protocollo vanno dichiarati in anticipo. Innanzitutto, si eseguono circa cinquanta confronti senza alcuna correzione per confronti multipli: le conclusioni si fondano perciò su intervalli ben lontani dallo zero, e nessuna differenza con un p-value prossimo a 0.05 viene considerata acquisita. Inoltre, il resampling dei dati non coglie la variabilità dovuta all'addestramento, che è invece documentata a parte dalle due esecuzioni del 0.5B (Sezione 6.2).

5.5 Latenza, energia e hardware

La latenza è misurata su 200 risposte di T1, 50 per ciascuna fascia di K, in modo che le risposte lunghe siano rappresentate quanto quelle brevi. GroundOne, il readout con una domanda per frase e tutti i giudici generativi girano in-process su MLX, una risposta alla volta, come farebbe un verificatore integrato in un assistente locale. Le prime tre risposte fungono da warm-up e sono escluse. Per le varianti System One ogni risposta è cronometrata tre volte e si conserva la mediana; i giudici generativi sono cronometrati una sola volta, dato che il greedy decoding è deterministico e ogni esecuzione dura molto di più. Il tempo riportato è il tempo wall-clock della chiamata al modello per una risposta, da cui si ricava il tempo per decisione.

Sono stati registrati i tempi anche per gli encoder, per Rizzo-Flow e per d1-3B, ma in condizioni che non permettono un confronto equo. Gli encoder giravano in PyTorch e valutavano ogni frase separatamente, rileggendo ogni volta il documento, il che li penalizza; Rizzo-Flow girava in un altro motore di inferenza, con pesi a 4 bit e dietro un'interfaccia HTTP; d1-3B girava in PyTorch con il proprio codice e con i riavvii dei worker descritti sopra. Nei loro confronti, quindi, non si avanza alcuna conclusione sulla latenza.

L'energia è misurata con `powermetrics`, lo strumento di macOS che riporta ogni 200 millisecondi la potenza assorbita da CPU, GPU e Neural Engine. Per ciascun sistema, 100 risposte del sottoinsieme di latenza (25 per fascia di K) vengono verificate una sola volta, dopo 60 secondi di misura a riposo con il modello già caricato; l'energia di una verifica è la potenza media durante l'esecuzione, al netto della potenza a riposo, moltiplicata per la durata dell'esecuzione e divisa per il numero di risposte. La misura comprende le unità di calcolo del chip ma non la memoria, lo schermo o l'archiviazione, ed è stata effettuata di notte, con la macchina altrimenti inattiva.

È in esecuzione un solo modello alla volta, e una guardia di memoria sorveglia ogni job lungo, registrando per tutta la sua durata la memoria libera e lo swap. I risultati vengono scritti una sola volta e mai sovrascritti, e ogni esecuzione conserva la propria configurazione: ogni numero del Capitolo 6 può così essere ricondotto all'esecuzione che lo ha prodotto.

Capitolo 6

Risultati

Questo capitolo illustra come GroundOne apprende e quanto è stabile il suo addestramento, quali risultati ottiene sui tre test set congelati, come si colloca rispetto alle baseline e quale contributo dia, di per sé, il readout del System One. Esamina poi la qualità delle probabilità, il tempo e l'energia necessari per verificare una risposta e gli errori che restano. Gli intervalli sono intervalli di confidenza al 95 % ottenuti con un bootstrap a cluster, calcolati come descritto nella Sezione 5.4.

6.1 Addestramento

Entrambi i modelli hanno appreso il task. Prima dell'addestramento, un modello che legge il formato con i marker non sa che cosa il marker gli chieda e i suoi verdetti sono casuali: il 0.5B risponde "contraddetta" per l'88 % delle frasi del dev set e raggiunge una balanced accuracy a tre classi di 0.32. Nelle esecuzioni completate la loss di addestramento è scesa, già nei primi cento passi, sotto il valore di un modello che conosca soltanto le frequenze delle classi ($\ln 3 \approx 1.10$), e il punteggio sul dev set è poi cresciuto con regolarità (Figura 6.1). Il 0.5B, addestrato su tutte le 13,203 risposte, ha toccato il punteggio migliore al passo 600 di 802, con una balanced accuracy di 0.633 in media sui tre task, per poi stabilizzarsi. Il 3B, addestrato per 12.25 ore su 6,500 risposte, ha raggiunto 0.705 al passo 320 di 403 sul proprio sottoinsieme del dev set. Sull'intero dev set la balanced accuracy a tre classi è di 0.731 per il 0.5B e di 0.784 per il 3B, e in entrambe le esecuzioni tutte e tre le classi sono rimaste presenti fra le predizioni per tutto l'addestramento.

6.2 Variabilità tra seed

La procedura del 0.5B è stata ripetuta con un secondo seed, che cambia l'inizializzazione degli adapter e l'ordine dei batch. Quando l'addestramento converge,

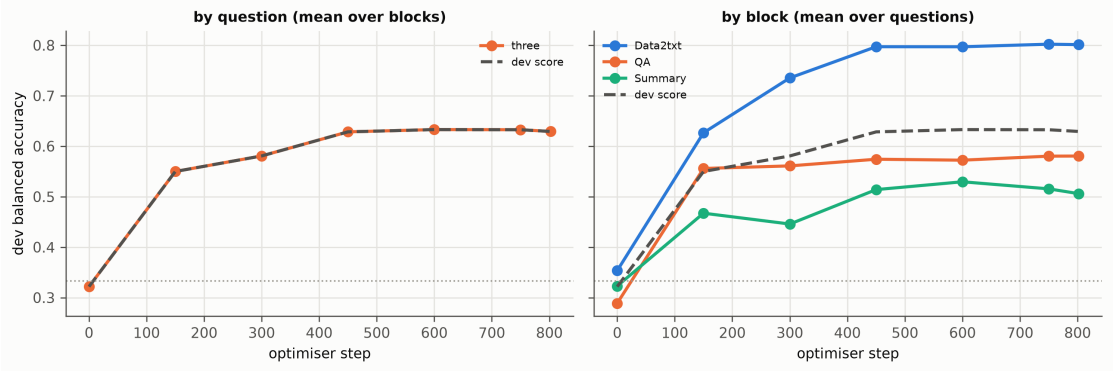


Figura 6.1: Balanced accuracy a tre classi sul dev set durante l'addestramento del modello 0.5B, complessiva (a sinistra) e per task (a destra). La linea punteggiata indica il livello casuale. Il 3B mostra lo stesso andamento sul suo sottoinsieme di 200 risposte del dev set.

converge allo stesso punto (Figura 6.2): la seconda esecuzione raggiunge su T1 una balanced accuracy a tre classi di 0.767 contro 0.769 e un AUROC di 0.907 contro 0.905, su LLM-AggreFact 71.5 contro 70.1 e su VitaminC 0.434 contro 0.442. Su T1 la differenza appaiata è +0.001 (da −0.008 a +0.011) e non è significativa. Le differenze emergono nei dettagli. Gli errori si distribuiscono in modo diverso tra le classi, con la recall delle frasi supportate e di quelle non menzionate che si sposta di circa 0.04 in direzioni opposte, e le porzioni più piccole dei dati oscillano di più: di 0.023 la fascia con quattordici o più decisioni, di 0.025 l'AUROC su VitaminC e fino a dieci punti i singoli dataset di LLM-AggreFact, come WiCE (67.9 contro 78.0).

Ne seguono due conseguenze. La prima è che il bootstrap, che ricampiona i dati di test, non coglie questa fonte di variabilità: su LLM-AggreFact e su VitaminC le due esecuzioni della stessa procedura differiscono più di quanto i loro intervalli consentirebbero (−0.013 sulla media macro di T2, −0.025 sull'AUROC di T3). Le differenze tra sistemi inferiori alla dispersione tra seed, cioè circa 1.5 punti sulla media macro di LLM-AggreFact e 0.025 di AUROC su VitaminC, non vengono pertanto interpretate, e lo stesso vale per i singoli dataset di LLM-AggreFact quando lo scarto non supera circa cinque punti.

La seconda è che l'addestramento non sempre converge. Un'esecuzione con un terzo seed è rimasta vicina al livello casuale: il suo punteggio sul dev set era 0.369 al passo 150 e 0.333 al passo 300, quando prediceva "supportata" per ogni frase, e la loss di addestramento è rimasta prossima a quella di un modello privo di informazione. L'esecuzione è stata interrotta al passo 470 di 801, quando il punteggio era risalito a 0.444; non è quindi possibile sapere se si sarebbe ripresa entro la fine dell'epoca, e in quanto incompleta è esclusa dalle tabelle. La procedura

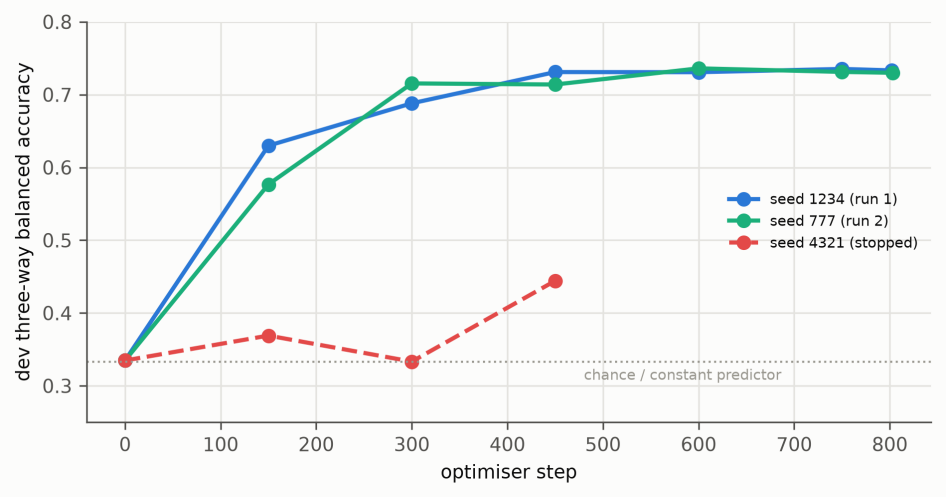


Figura 6.2: Balanced accuracy a tre classi sul dev set del modello 0.5B durante l'addestramento, con la stessa procedura e tre seed. Due esecuzioni convergono allo stesso livello; la terza è collassata su un predittore costante ed è stata interrotta.

ha due caratteristiche note per rendere fragile il fine-tuning: non applica gradient clipping e l'ottimizzatore lavora senza bias correction, l'impostazione predefinita del framework, che è stata indicata come una delle cause principali dei fine-tuning non riusciti [60, 59]. Un solo collasso su tre esecuzioni non basta a stimare un tasso di fallimento, ma mostra che la procedura non è del tutto robusta rispetto all'inizializzazione.

6.3 Accuratezza sul test set RAG (T1)

Sulle 17,284 frasi di T1 il modello 0.5B raggiunge una balanced accuracy a tre classi di 0.769 e il 3B di 0.832 (Tabella 6.1); la differenza, +0.063 (da +0.052 a +0.076), è l'effetto più netto della dimensione del modello. Sulla decisione binaria il 0.5B ottiene una balanced accuracy di 0.815 e un AUROC di 0.905, il 3B rispettivamente 0.874 e 0.940. Entrambi superano di gran lunga i segnali che non confrontano mai una frase con il suo documento: la sovrapposizione lessicale arriva a un AUROC di 0.737 e il classificatore che vede la sola frase a una balanced accuracy a tre classi di 0.570.

Spiccano tre aspetti. Il primo è che l'accuratezza non peggiora con la lunghezza della risposta: le risposte con quattordici o più decisioni sono tra quelle gestite meglio, con 0.723 per il 0.5B e 0.839 per il 3B, mentre la fascia più difficile è quella delle risposte più brevi, che comprende soprattutto risposte di question answering e di summarization. Il dato è importante perché è proprio sulle risposte

Tabella 6.1: GroundOne su T1, per task e per numero di decisioni per risposta (K): balanced accuracy a tre classi, recall delle frasi contraddette e AUROC di P(supported).

		GroundOne 0.5B			GroundOne 3B		
Sottoinsieme	Frasi	Bal. acc.	Contr.	AUROC	Bal. acc.	Contr.	AUROC
Tutte	17,284	0.769	0.68	0.905	0.832	0.76	0.940
Question answering	4,876	0.589	0.03	0.916	0.699	0.30	0.948
Summarization	4,538	0.544	0.13	0.775	0.629	0.30	0.873
Data-to-text	7,870	0.812	0.84	0.929	0.873	0.90	0.955
$K = 1$–4	2,434	0.606	0.18	0.857	0.677	0.25	0.921
$K = 5$–8	8,012	0.758	0.67	0.899	0.829	0.78	0.936
$K = 9$–13	5,201	0.824	0.82	0.926	0.863	0.86	0.945
$K \geq 14$	1,637	0.723	0.50	0.908	0.839	0.68	0.957

lunghe che un System One fa risparmiare più tempo; su di esse il guadagno del 3B rispetto al 0.5B è di +0.116 (da +0.050 a +0.180). Il secondo è che le contraddizioni vengono riconosciute soprattutto nel data-to-text, dove RAGTruth le concentra. Nel question answering e nella summarization il 0.5B distingue bene le frasi supportate da quelle non menzionate, ma non individua quasi nessuna contraddizione, con una recall di 0.03 e 0.13. Il 3B porta entrambe a 0.30: la dimensione del modello compensa in parte la scarsità di questi esempi nei dati di addestramento. Il terzo è che question answering e summarization offrono la prova più chiara che i modelli leggono davvero il documento: qui il classificatore che vede la sola frase resta al livello casuale (0.36 e 0.35), mentre i due modelli raggiungono 0.59–0.70 e 0.54–0.63.

Le stesse predizioni si possono leggere anche a livello di risposta intera, che è il criterio con cui il Decision Index della comunità valuta i decision model generalisti su RAGTruth: una risposta è allucinata se contiene un qualsiasi contenuto non supportato, e la metrica è l'F1 di questa classe, che un sistema che risponda sempre "allucinata" raggiunge già con 0.52 [28]. Prendendo il minimo della P(supported) calibrata sulle frasi di una risposta, con una soglia stimata sul dev set, GroundOne raggiunge 0.733 con il 0.5B e 0.784 con il 3B sulle 2,647 risposte di T1. Nella leaderboard nessun decision model generalista sotto il miliardo di parametri supera 0.55, Jev raggiunge 0.765 e il modello migliore, da 27B parametri, 0.803. Il confronto è solo indicativo, perché GroundOne è stato addestrato su RAGTruth, mentre i modelli della leaderboard sono valutati zero-shot, con un prompt diverso e su 53 risposte in più; colloca comunque uno specialista da 0.5B molto al di sopra dei decision model generalisti della stessa dimensione.

6.4 Confronto con le baseline

La Figura 6.3 mette a confronto tutti i sistemi su T1. Gli encoder specializzati raggiungono un AUROC di 0.814 (MiniCheck-RoBERTa) e 0.821 (HHEM), superiore a quello della sovrapposizione lessicale ma inferiore a quello di entrambi i modelli GroundOne: il 0.5B li supera di +0.091 (da +0.074 a +0.107) e +0.083 (da +0.066 a +0.099), il 3B di +0.126 e +0.119. Il confronto per task ridimensiona però questo risultato. Il vantaggio di GroundOne viene in gran parte dal data-to-text, dove è addestrato nel dominio e dove gli encoder sono più deboli (AUROC 0.71–0.75), mentre nella summarization MiniCheck-RoBERTa supera il modello 0.5B (0.872 contro 0.775) ed è alla pari con il 3B (0.873). I modelli non addestrati mostrano quanta parte dell'accuratezza si debba all'addestramento: il 0.5B è al livello casuale perché non risponde mai "supportata", e il 3B, a cui si pone la stessa domanda, raggiunge 0.437 contro 0.818 dopo l'addestramento, sulle stesse risposte. Il contributo del readout ristretto, al di là dell'addestramento, è misurato separatamente nella Sezione 6.6.

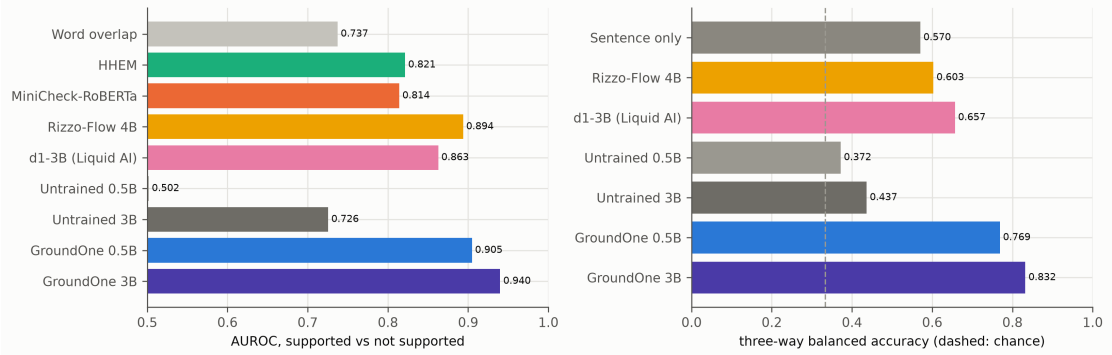


Figura 6.3: T1: AUROC di P(supported) per ogni sistema (a sinistra) e balanced accuracy a tre classi per i sistemi con output a tre classi (a destra). Rizzo-Flow è valutato su 300 risposte e il 3B non addestrato su 600; tutti gli altri sistemi, compreso il decision model generalista d1-3B, sull'intero test set.

Il confronto più istruttivo è quello con il System One generalista. Sulle stesse 300 risposte di T1, Rizzo-Flow, un modello da 4B sottoposto a fine-tuning per decisioni generiche, raggiunge una balanced accuracy a tre classi di 0.603, contro 0.798 del GroundOne 0.5B e 0.840 del 3B; le differenze, +0.195 (da +0.136 a +0.249) e +0.237 (da +0.184 a +0.285), sono ampie. Il suo AUROC (0.894) è vicino a quello del 0.5B su queste risposte (0.916; la differenza non è significativa), perché ordina bene le frasi ma predice "supportata" per il 90 % di esse; il 3B le ordina meglio (+0.056, da +0.034 a +0.079). Uno specialista otto volte più piccolo supera quindi

il generalista nella decisione per cui è stato progettato, e lo eguaglia nella capacità di ordinare le frasi.

Un decision model generalista della stessa dimensione offre la verifica più severa del valore della specializzazione, perché d1-3B è già addestrato a prendere decisioni tipizzate e si differenzia da GroundOne soprattutto per i dati su cui è stato addestrato. Sull'intero T1 raggiunge una balanced accuracy a tre classi di 0.657 e un AUROC di 0.863. Entrambi i modelli GroundOne fanno nettamente meglio: il 0.5B, sei volte più piccolo, di +0.112 (da +0.092 a +0.133) nella balanced accuracy a tre classi e di +0.042 in AUROC, il 3B di +0.175 (da +0.157 a +0.195) e di +0.077. Il divario è massimo sulle risposte lunghe, dove d1-3B scende a 0.527 con quattordici o più decisioni mentre il GroundOne 3B resta a 0.839 (+0.312, da +0.230 a +0.405); inoltre d1-3B riconosce soltanto metà delle frasi non menzionate (recall 0.49). Nel dominio delle risposte RAG, dunque, la specializzazione vale molto più di un addestramento generico a prendere decisioni.

Su LLM-AggreFact (T2) il modello 0.5B raggiunge una balanced accuracy media macro di 70.1 sugli undici dataset, contro 64.2 dello stesso modello usato zero-shot e 72.8 del modello 7B zero-shot nella leaderboard pubblica. Resta sotto MiniCheck-RoBERTa (74.0) di 3.9 punti, una differenza significativa. Sui quattro dataset più vicini alla retrieval-augmented generation i due sono alla pari (74.4 contro 74.6), e nella colonna di RAGTruth GroundOne è in vantaggio, 82.1 contro 76.5, in un confronto che lo favorisce perché è stato addestrato sulle risposte del training set di RAGTruth. Al di fuori del RAG il 0.5B è inferiore all'encoder di sette-nove punti su AggreFact-CNN, AggreFact-XSum, FactCheck-GPT e REVEAL: è il costo atteso di un addestramento sui soli dati RAG.

Il modello 3B, valutato sull'intero test set, colma gran parte di questo divario. La sua balanced accuracy media macro è 74.8, alla pari con MiniCheck-RoBERTa (74.0; differenza appaiata +0.8 punti, intervallo da −0.5 a +2.1) e superiore al punteggio zero-shot pubblicato per il modello 7B (72.8, per il quale non si può calcolare un intervallo). Sui dataset RAG raggiunge 76.9, 2.4 punti sopra l'encoder (intervallo da +1.4 a +3.3). Il suo guadagno rispetto al 0.5B, +4.7 punti (da +3.4 a +5.9), è diffuso, con i miglioramenti maggiori su REVEAL, WiCE, FactCheck-GPT e XSum: la dimensione del modello migliora quindi il trasferimento al di fuori del dominio di addestramento. Il 3B resta nettamente sotto l'encoder soltanto nella summarization di notizie (AggreFact-CNN, 59.3 contro 67.1). Rizzo-Flow, sui 1,093 elementi del suo sottocampione, raggiunge 61.5, e su questi il GroundOne 3B fa meglio di 12.0 punti (da +7.4 a +16.5). Una valutazione indipendente del System One closed Jev sullo stesso test set e con la stessa metrica riporta 78.6, e 76.9 per un modello open da 27B letto attraverso le lettere di risposta, con prompt diversi da quelli usati qui, per cui non è possibile calcolare alcun intervallo [21]. Su un benchmark ampio di groundedness, quindi, uno specialista da 3B addestrato soltanto su risposte RAG eguaglia il miglior encoder di piccole dimensioni, ma resta

sotto lo stato dell'arte closed e sotto modelli open molto più grandi. Il generalista d1-3B conferma il quadro a parità di dimensione: su un sottocampione casuale di 4,000 elementi raggiunge 75.7, contro 74.3 del GroundOne 3B sugli stessi elementi, una differenza non significativa (-1.3, da -4.0 a $+1.0$), mentre supera il 0.5B di 6.5 punti. È più debole nella colonna di RAGTruth (78.9 contro 87.3) e nettamente più forte su WiCE e sulla summarization di dialoghi.

6.5 Trasferimento a un dominio diverso (T3)

Su VitaminC il quadro si rovescia. Sull'intero test set il 0.5B raggiunge una balanced accuracy a tre classi di 0.442 e il 3B di 0.527, con AUROC di 0.722 e 0.817, mentre MiniCheck-RoBERTa arriva a un AUROC di 0.968. Sulle 600 coppie valutate con Rizzo-Flow, il modello generalista ottiene 0.712 e un AUROC di 0.909, contro 0.442 e 0.744 del 0.5B e 0.537 e 0.845 del 3B. Due fatti aiutano a interpretare questi numeri. MiniCheck-RoBERTa parte dal modello AlignScore, i cui dati di addestramento comprendono VitaminC [38]: T3 appartiene quindi alla sua distribuzione di addestramento, mentre per GroundOne misura il trasferimento da RAGTruth. Inoltre, il guadagno del 3B sul 0.5B in T3 si deve alla dimensione del modello più che all'addestramento su dati RAG: il 3B non addestrato raggiunge già 0.531 e un AUROC di 0.821. Il decision model generalista della stessa dimensione rende esplicito il contrasto: d1-3B raggiunge 0.724 e un AUROC di 0.913 sull'intero T3, davanti al GroundOne 3B di 0.197 e 0.096, l'esatto opposto di quanto accade su T1. La specializzazione su risposte RAG migliora quindi in modo sostanziale il task RAG, ma non basta, da sola, a produrre un fact checker generale. La Sezione 6.11 mostra quali casi sfuggono.

6.6 Readout e generazione a confronto

Il controllo B2 isola il contributo del readout. A parità di pesi ottenuti con il fine-tuning, di prompt e di singolo forward pass, il verdetto a ogni marker viene letto dalle tre lettere di risposta, come fa GroundOne, oppure preso come il token che emetterebbe la generazione greedy (Figura 6.4). Per il 0.5B il token generato dà una balanced accuracy a tre classi di 0.521 contro 0.769 del readout, una differenza di $+0.248$ (da $+0.228$ a $+0.269$): in un quarto dei marker (24.7 %) il modello non emetterebbe affatto una lettera, ma per lo più un semplice spazio, l'inizio di una risposta più lunga. Per il 3B il token generato è quasi sempre una lettera (1.2 % di errori di formato) e concorda con il readout nel 92 % dei marker, eppure il readout resta migliore, 0.832 contro 0.761 ($+0.071$, da $+0.059$ a $+0.083$). Sulle risposte con quattordici o più decisioni il divario è massimo per il modello piccolo, $+0.346$.

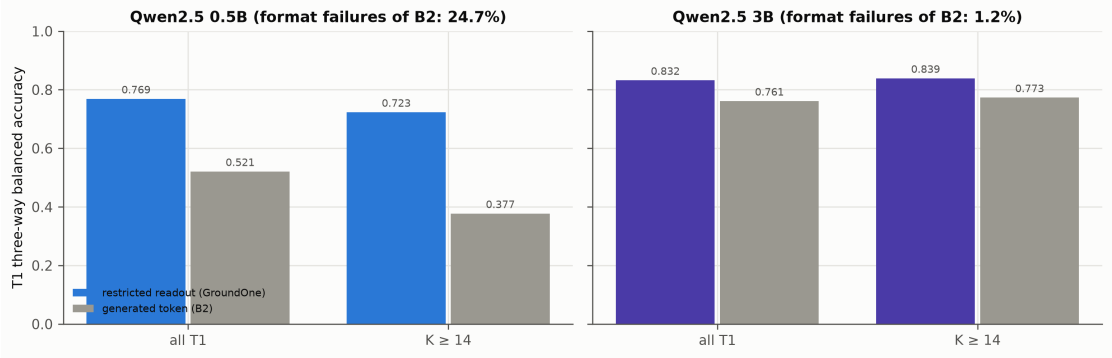


Figura 6.4: Controllo B2 sull'intero T1: con gli stessi pesi e lo stesso forward pass, il verdetto è letto dalle tre lettere di risposta (GroundOne) oppure preso come il token che emetterebbe la generazione greedy (gli errori di formato sono contati come errori).

Le due opzioni hanno la stessa latenza, perché entrambe richiedono un solo forward pass; a cambiare è la regola di decisione. Restringere la scelta alle opzioni ammissibili elimina per costruzione gli errori di formato e, poiché il modello è addestrato soltanto a ordinare le tre lettere, fornisce anche la decisione più accurata (Sezione 3.2). Più il modello è piccolo, più questa restrizione conta. L'idea, spesso ripetuta, che leggere i logit delle risposte e generare un singolo token di risposta siano equivalenti vale dunque per il costo, ma non per l'accuratezza di un verificatore sottoposto a fine-tuning.

6.7 Calibrazione e selective prediction

Le probabilità grezze sottostimano il supporto. Nel reliability diagram (Figura 6.5) la curva grezza giace sopra la diagonale, perché i pesi delle classi addestrano il modello come se le classi fossero bilanciate; l'ECE binario su T1 è 0.30 per il 0.5B e 0.18 per il 3B. La temperatura e il bias stimati sul dev set portano l'ECE su T1 a 0.019 per entrambi i modelli, e a 0.020 per la seconda esecuzione del 0.5B, e riducono la negative log-likelihood da 0.55 a 0.21 (0.5B) e da 0.36 a 0.17 (3B), lasciando l'AUROC praticamente invariato. La mappa stimata sul dev set si trasferisce quindi al test set dello stesso dominio.

La calibrazione ha però una conseguenza da tenere presente. Il bias ripristina la frequenza reale di ciascuna classe, per cui la classe calibrata più probabile diventa più spesso "supportata": per il 0.5B su T1 l'accuratezza sale da 0.82 a 0.93, ma la balanced accuracy a tre classi scende da 0.77 a 0.63. Scegliere la classe calibrata più probabile non è quindi una buona regola di decisione quando i tre tipi di errore hanno lo stesso peso. La regola con cui vengono restituite le etichette, descritta

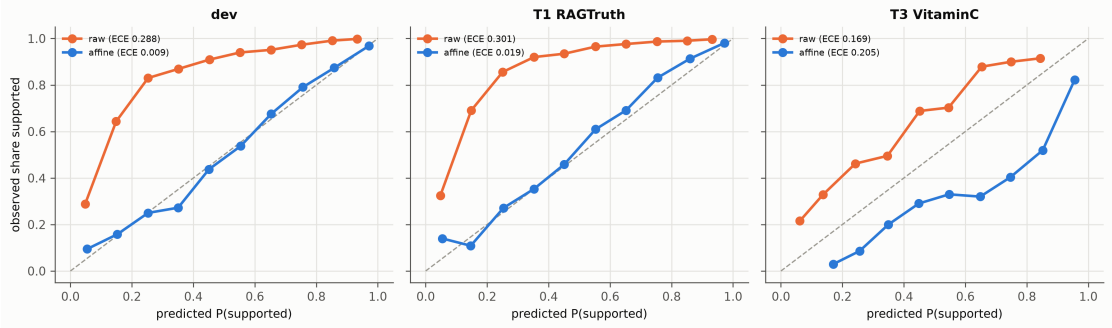


Figura 6.5: Reliability diagram di $P(\text{supported})$ del 0.5B prima (grezza) e dopo la calibrazione sul dev set, per il dev set, T1 e T3. I punti corrispondono a intervalli con almeno 20 frasi.

nella Sezione 3.4, risolve il problema: con la soglia stimata sul dev set, $\tau = 0.935$ per il 0.5B, 0.895 per la sua seconda esecuzione e 0.925 per il 3B, raggiunge 0.758, 0.769 e 0.830 su T1, entro 0.01 dalla decisione grezza valutata in tutto il capitolo, e mantiene coerenti etichette e probabilità calibrate. Le soglie sono alte perché il 91 % delle frasi di RAGTruth è supportato: una frase viene restituita come supportata soltanto quando il modello è molto sicuro.

Usate come confidenze, le probabilità calibrate rendono efficace la selective prediction: rinviare le frasi su cui il modello è meno sicuro riduce rapidamente il tasso di errore della decisione binaria, con un'area sotto la curva risk–coverage di 0.015 (0.5B) e 0.010 (3B) su T1 (Figura 6.6).

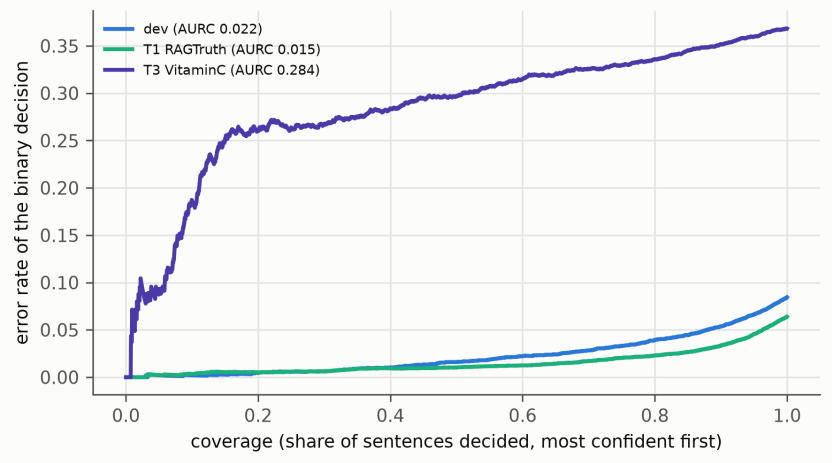


Figura 6.6: Curve risk–coverage della decisione binaria del 0.5B con probabilità calibrate: tasso di errore al crescere delle frasi meno sicure che vengono rinviate.

La calibrazione non si trasferisce a VitaminC, dove l'ECE binario sale da 0.17 a 0.21 per il 0.5B e da 0.14 a 0.19 per il 3B, e dove la regola di restituzione perde 0.02–0.04 rispetto alla decisione grezza. Il bias codifica il tasso di base di RAGTruth, mentre VitaminC è bilanciato per costruzione. Una calibrazione vale per la distribuzione su cui è stata stimata, per cui ogni impiego reale richiede un piccolo campione etichettato dei propri dati.

6.8 Probabilità a parità di output

I giudici a parità di output servono a stabilire se un giudice generativo della stessa dimensione, a cui si chiede lo stesso contenuto di GroundOne, possa eguagliare la qualità delle sue probabilità. La risposta è negativa (Figura 6.7). Sulle 200 risposte del sottoinsieme di latenza, con tutti i sistemi calibrati allo stesso modo sul dev set, l'AUROC di $P(\text{supported})$ è 0.94 e 0.97 per GroundOne, 0.48 e 0.68 per il giudice le cui probabilità sono lette dai logit, e 0.47 e 0.58 per il giudice che le scrive, rispettivamente per il 0.5B e per il 3B. L'area sotto la curva risk–coverage è 0.009 e 0.005 per GroundOne, contro valori da 0.048 a 0.063 per i giudici, da cinque a dodici volte maggiori. L'accuratezza delle decisioni segue lo stesso andamento: 0.835 e 0.900 per GroundOne, al massimo 0.41 per qualsiasi giudice.

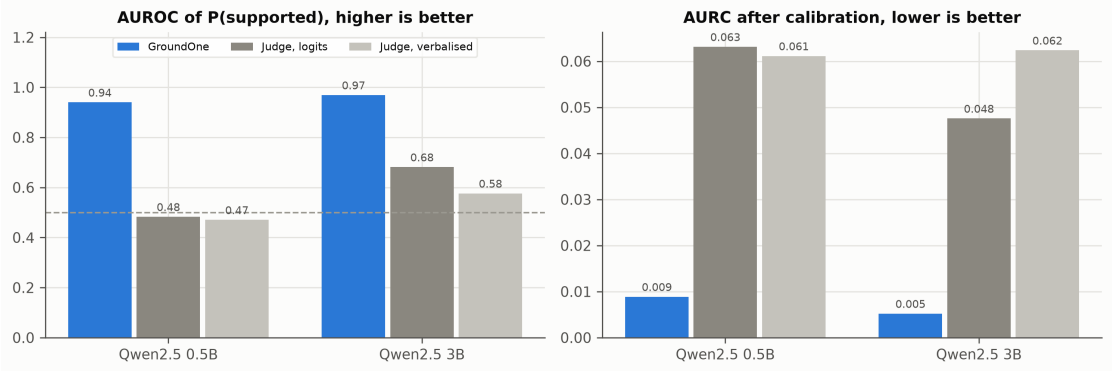


Figura 6.7: Confronto a parità di output sulle 200 risposte di latenza: giudici generativi della stessa dimensione a cui si chiede l'output di GroundOne, con probabilità lette dai logit della lettera generata oppure scritte in JSON; tutti i sistemi sono calibrati sul dev set con la stessa mappa. Solo frasi con un verdetto valido.

Altre due osservazioni contano nella pratica. Le probabilità scritte a parole non portano quasi alcuna informazione: il 3B scrive un vettore con tutta la massa su una sola classe per il 93 % delle frasi, usando appena sedici vettori distinti, e il 0.5B scrive pareggi non informativi, come probabilità uguali per tutte le classi;

contando come errori i pareggi privi di etichetta, la sua balanced accuracy scende da 0.234 a 0.081. Il dato concorda con l'osservazione che la confidenza verbalizzata dei modelli piccoli è grossolana e poco informativa [53, 52]. Inoltre, l'ECE da solo nasconderebbe tutto ciò: dopo la calibrazione ogni sistema, giudici compresi, ottiene un ECE binario compreso tra 0.03 e 0.05, perché una mappa di calibrazione può allineare la confidenza media al tasso di base anche quando le probabilità non separano le classi. La capacità discriminativa va misurata a parte, con l'AUROC e con la curva risk–coverage.

Chiedere le probabilità, infine, ha un costo. Il giudice che le legge dai logit è veloce quanto il giudice sintetico, perché genera le stesse lettere, mentre il giudice che scrive l'oggetto JSON impiega sul 3B 17.0 secondi per risposta con quattordici o più decisioni, quanto il giudice verboso, per un'accuratezza non superiore a quella del giudice sintetico.

6.9 Latenza in funzione di K

La Figura 6.8 riporta la misura centrale della tesi: il tempo necessario per verificare una risposta, sulle stesse 200 risposte di T1, al crescere del numero di decisioni. Tra i sistemi eseguiti in-process su MLX, GroundOne è il più veloce a entrambe le dimensioni e in ogni fascia di K, e il suo vantaggio aumenta con K. Per le risposte con quattordici o più decisioni il GroundOne 0.5B impiega 0.41 secondi, contro 0.80 del giudice sintetico e 2.51 del giudice verboso della stessa dimensione: è 1.95 volte più veloce del primo (da 1.82 a 2.14) e 6.16 volte più veloce del secondo (da 5.72 a 6.67). Il GroundOne 3B impiega 2.57 secondi, contro 4.44 e 15.86, ed è quindi 1.73 volte (da 1.65 a 1.82) e 6.18 volte (da 5.88 a 6.65) più veloce. Leggere tutti i verdetti in un unico forward pass è anche più rapido che porre una domanda per frase in un batch, di un fattore 1.4–1.5 ai valori più alti di K. L'altra leva è la dimensione del modello: il GroundOne 0.5B verifica una risposta circa sei volte più in fretta del 3B (0.33 contro 1.99 secondi in mediana).

La velocità non è l'unica differenza (Figura 6.9). I giudici generativi non addestrati sono anche molto meno accurati e non sempre rispettano il formato di output. Il giudice verboso 0.5B produce un verdetto interpretabile soltanto per il 3 % delle frasi, perché per lo più ricopia la risposta, e il giudice sintetico 0.5B fallisce sul 23 %. I giudici 3B rispettano il formato molto meglio (0.1 % e 7.6 % di errori di formato), ma si fermano a una balanced accuracy a tre classi di circa 0.40, contro 0.90 del GroundOne 3B sulle stesse risposte. Il giudice verboso 3B scrive in media 248 token per risposta, che vanno tutti generati prima di poter usare il verdetto, mentre GroundOne non ne genera nessuno.

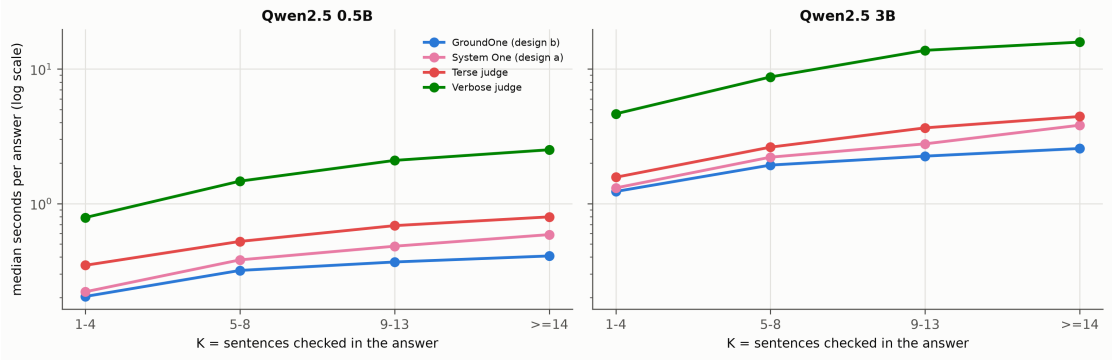


Figura 6.8: Tempo mediano per risposta in funzione di K sulle stesse 200 risposte di T1 (MLX in-process, una risposta alla volta, M4 con 16 GB), per GroundOne, il System One con una domanda per frase e i giudici generativi sintetico e verboso. Scala logaritmica.

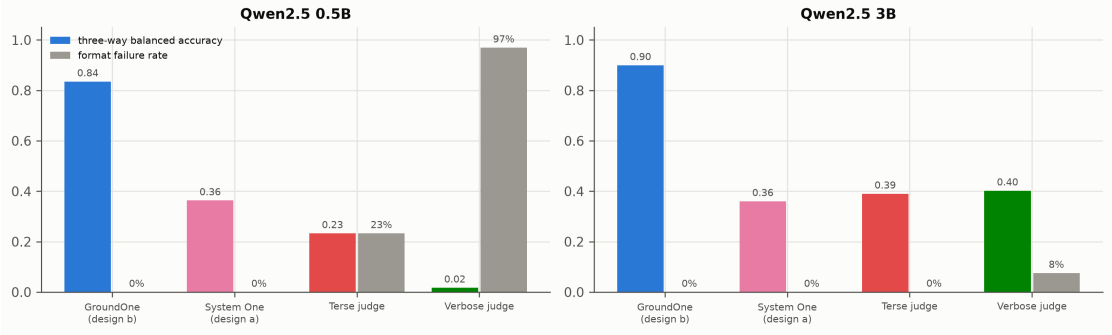


Figura 6.9: Sul sottoinsieme di latenza: balanced accuracy a tre classi (gli errori di formato sono contati come errori) e quota di frasi il cui verdetto non è stato possibile interpretare.

6.10 Energia per verifica

La misura dell'energia introduce una precisazione che la sola latenza nasconde (Figura 6.10). GroundOne è il sistema che consuma meno energia a entrambe le dimensioni: 5.5 joule per risposta con il 0.5B e 33 con il 3B, contro 8.9 e 43 del giudice sintetico e 22.5 e 112 del giudice verboso. Il suo vantaggio, tuttavia, è minore di quello sul tempo: 1.6 e 1.3 volte meno energia del giudice sintetico, 4.1 e 3.4 volte meno del verboso. Il motivo è la potenza assorbita durante il calcolo. GroundOne trascorre il suo tempo in un unico, lungo prefill, che tiene la GPU pienamente occupata e assorbe circa 16 W oltre la potenza a riposo, mentre il decoding autoregressivo è limitato dalla banda di memoria e assorbe 11–14 W, ma per un tempo molto più lungo. È il comportamento atteso dalle due fasi

dell'inferenza dei transformer [10], e implica che uno speed-up non vada mai letto come un risparmio di energia equivalente [16, 17].

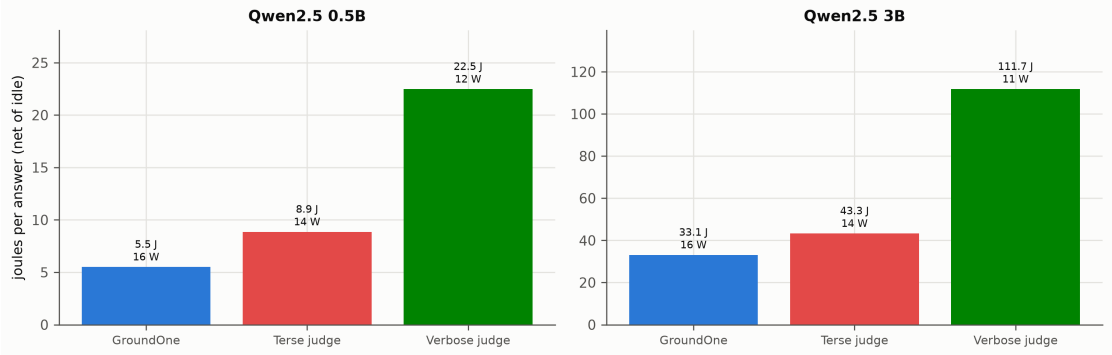


Figura 6.10: Energia per risposta verificata su 100 risposte di latenza (powermetrics, CPU, GPU e Neural Engine, al netto della potenza a riposo), con la potenza netta media durante l'esecuzione.

In termini assoluti i valori sono piccoli: 5.5 joule corrispondono a circa 0.0015 wattora, per cui il GroundOne 0.5B potrebbe verificare circa seicento risposte con l'energia di un wattora, interamente sul portatile.

6.11 Analisi degli errori

Il risultato debole su VitaminC e le contraddizioni mancate su T1 hanno una causa comune, che la sovrapposizione lessicale mette in evidenza. Su VitaminC, quando un'affermazione supportata condivide poche parole con la propria evidenza (sovrapposizione inferiore a 0.4), il 0.5B la riconosce soltanto nel 2 % dei casi, contro il 64 % quando la sovrapposizione supera 0.7, e non riconosce mai una contraddizione in un'affermazione priva di cifre. I modelli si appoggiano in parte alla sovrapposizione lessicale: un'affermazione parafrasata sembra "non menzionata", mentre una piccola modifica a una frase copiata, spesso un numero, passa inosservata.

La stessa debolezza si ritrova su T1, con un peso minore. Quando una frase condivide con il documento poche o alcune parole, la balanced accuracy a tre classi del 0.5B è 0.77, ma per le frasi che riprendono dal documento più del 70 % delle proprie parole scende a 0.54, con una recall di 0.24 per le frasi contraddette e di 0.43 per quelle non menzionate. Il 3B riduce questo punto cieco senza eliminarlo: sulle stesse frasi la sua recall è 0.36 e 0.56, e la sua balanced accuracy 0.63. Le allucinazioni nascoste in frasi che per il resto ricopiano la fonte restano il limite principale di entrambi i modelli. VitaminC non è più difficile in assoluto: è composto in gran parte proprio da casi di questo tipo, supporto parafrasato e

modifiche minime, che in RAGTruth sono meno numerosi. L'Appendice A mostra l'output di ogni sistema sulla stessa risposta.

Capitolo 7

Discussione, limiti e minacce alla validità

Questo capitolo interpreta i risultati alla luce delle cinque domande di ricerca, quindi indica ciò che gli esperimenti non possono dimostrare e quali scelte potrebbero averli distorti.

7.1 Quando conviene un System One specialista

Specializzazione (RQ1). A rendere GroundOne accurato è l'addestramento sulla decisione. I modelli non addestrati restano vicini al caso, mentre gli stessi modelli, una volta addestrati, raggiungono sul test set di RAGTruth una balanced accuracy a tre classi di 0.769 e 0.832, superando in AUROC gli encoder specializzati e, in accuratezza a tre classi, un System One generalista otto volte più grande. Il vantaggio si concentra dove si trovano i dati di addestramento: su LLM-AggreFact il 3B si porta alla pari con il miglior encoder di piccole dimensioni, senza superarlo, e resta sotto il System One chiuso Jev e un modello aperto da 27B, misurati in modo indipendente; il 0.5B rimane al di sotto dell'encoder e, su VitaminC, il 3B addestrato non fa meglio di quello non addestrato. Il confronto con un decision model generalista della stessa dimensione rende esplicito il compromesso. d1-3B, già addestrato a decidere ma su un'ampia miscela di task, è molto lontano da GroundOne su RAGTruth, soprattutto sulle risposte lunghe, alla pari con il 3B su LLM-AggreFact e nettamente superiore su VitaminC. Uno specialista è dunque una scelta solida per il dominio su cui è stato addestrato, ma uno strumento debole per la verifica dei fatti in generale: la specializzazione garantisce accuratezza e robustezza sulle risposte lunghe all'interno del dominio a scapito del trasferimento, mentre fuori dal dominio sono la dimensione del modello, oppure un'ampia miscela di addestramento, a migliorare il trasferimento.

Readout e generazione a confronto (RQ2). Il meccanismo System One contribuisce in due modi distinti. Il primo riguarda il costo: un solo forward pass sulla risposta sostituisce la generazione di K verdetti, per cui il tempo risparmiato cresce con la lunghezza della risposta, fino a un fattore di circa due rispetto a un giudice sintetico e di sei rispetto a uno verboso per risposte di quattordici o più frasi. Il secondo, che questa tesi non aveva previsto, riguarda l'accuratezza. A parità di pesi e di costo, scegliere tra le tre lettere ammissibili è più accurato che prendere il token che il modello genererebbe, con un guadagno di 0.25 per il 0.5B e di 0.07 per il 3B. Un System One sottoposto a fine-tuning si comprende meglio come un classificatore a tre classi innestato su un modello linguistico che come un generatore che si ferma dopo un token; la distinzione pesa soprattutto per i modelli piccoli, cioè proprio quelli che un portatile può permettersi. Il confronto con i giudici generativi, infine, misura anche l'effetto dell'addestramento: poiché i giudici non sono addestrati, le ampie differenze di accuratezza rispetto a GroundOne vanno lette come l'effetto congiunto di specializzazione e readout.

Calibrazione (RQ3). Una temperatura e un bias stimati sul dev set allineano le probabilità alle frequenze osservate sul test set dello stesso dominio, e le probabilità calibrate rendono efficace la selective prediction. Due indicazioni valgono al di là di GroundOne. In primo luogo, l'Expected Calibration Error da solo non basta a ordinare i sistemi: dopo la stessa calibrazione, giudici le cui probabilità separano a malapena le classi raggiungono lo stesso ECE di GroundOne, e soltanto l'AUROC e la risk–coverage curve ne rivelano la differenza. In secondo luogo, decisioni e probabilità vanno ricavate in modo coerente: la classe calibrata più probabile collassa verso la classe maggioritaria, mentre una soglia stimata sul dev set recupera la decisione bilanciata e mantiene l'etichetta coerente con la probabilità mostrata all'utente. La calibrazione, come mostrano i risultati su VitaminC, vale soltanto per la distribuzione su cui è stata stimata.

Costo sul dispositivo (RQ4). Il verificatore sta comodamente nel portatile da 16 GB: l'addestramento ha picchi di 2.6 e 5.9 GB, e verificare una risposta lunga richiede 0.41 secondi con il 0.5B e 2.57 con il 3B, con un consumo di 5.5 e 33 joule. Il 0.5B rientra in un budget interattivo; il 3B, su una risposta tipica, è più lento di quanto un uso interattivo richiederebbe e scambia tempo con accuratezza. Le misure di energia precisano la portata dei risultati sulla velocità: un singolo prefill lungo assorbe più potenza del decoding token per token, per cui l'energia risparmiata è inferiore al tempo risparmiato, e la maggiore velocità non va letta come un risparmio equivalente di batteria. Non è stata misurata la memoria necessaria quando il verificatore gira accanto a un generatore e a un retriever.

Oltre un punteggio binario (RQ5). L'output a tre classi distingue bene le frasi contraddette da quelle non menzionate nel data-to-text, dove RAGTruth concentra le proprie contraddizioni, e molto meno nel question answering e nella summarization, dove nei dati di addestramento le contraddizioni sono rare. Il 3B migliora sensibilmente su questo fronte, il che suggerisce che al modello piccolo manchino, più che un'architettura diversa, contraddizioni più varie in addestramento. Poiché le policy strict e lenient e la soglia derivano dalle stesse tre probabilità, l'utente può cambiare la nozione di errore senza riaddestrare il modello, cosa che nessun verificatore binario consente.

7.2 Limiti

Una sola piattaforma e una sola famiglia di modelli. Tutti i risultati provengono da un unico portatile, un unico runtime e un'unica famiglia di modelli. Latenza ed energia dipendono dall'hardware e dall'implementazione: ciò che dovrebbe valere anche altrove sono i rapporti tra i metodi, non i valori assoluti. Non si confronta la latenza con gli encoder né con il System One generalista, perché questi sono stati eseguiti in runtime diversi e in condizioni che renderebbero il confronto iniquo.

Stabilità dell'addestramento. Due run completi del 0.5B concordano su tutte le metriche aggregate, ma un terzo run è collassato nelle prime fasi ed è stato interrotto. La ricetta di addestramento non prevede gradient clipping e usa Adam senza bias correction, due cause note di fine-tuning instabile [60, 59]: l'una o l'altra potrebbe spiegare il collasso, ma nessuna delle due ipotesi è stata verificata. Il 3B è stato addestrato una sola volta, su metà delle risposte di addestramento, e la sua variabilità al cambiare del seed è ignota.

Dati di addestramento. RAGTruth è l'unica fonte di addestramento. Le sue contraddizioni sono concentrate nel data-to-text e, al di fuori di questo task, vengono apprese solo in parte. I modelli si affidano inoltre in parte alla sovrapposizione lessicale: quando il supporto è espresso con parole diverse la frase appare non supportata, e una piccola modifica in una frase che per il resto copia la fonte passa spesso inosservata. Questo punto cieco si riduce al crescere del modello, ma resta il limite principale di entrambi.

Ambito della decisione. La verifica avviene frase per frase e soltanto rispetto al documento recuperato. Una frase vera nel mondo ma assente dal documento conta come non menzionata: è il comportamento corretto per la groundedness, non per la verifica dei fatti. Le frasi prive di contenuto verificabile vengono escluse

da una regola, e la segmentazione stessa può dividere o accorpare le affermazioni diversamente da un lettore umano. Il verdetto su una frase dipende soltanto dal documento e dalle frasi che la precedono.

Portata statistica. Gli intervalli bootstrap descrivono la variabilità del test set, non quella dell'addestramento, e circa cinquanta confronti sono stati sottoposti a test senza correzione per confronti multipli. Le conclusioni poggiano quindi su effetti ben lontani dallo zero, e non si rivendicano differenze su LLM-AggreFact e VitaminC inferiori alla dispersione tra seed. Diverse baseline sono state valutate su sottocampioni, il che ne allarga gli intervalli.

7.3 Minacce alla validità

Etichette. La distinzione tra frase contraddetta e non menzionata si basa sugli annotatori di RAGTruth, il cui accordo sul tipo di allucinazione non è pubblicato; inoltre, una recente riannotazione della partizione di test di RAGTruth ha individuato un numero di risposte allucinate considerevolmente superiore a quello indicato dalle etichette originali [76]. Il test set potrebbe quindi sottostimare le classi minoritarie. Per questo i risultati a tre classi su RAGTruth vanno sempre letti insieme a quelli su VitaminC, le cui etichette hanno un accordo misurato. LLM-AggreFact unifica le etichette di undici dataset di qualità diversa, e alcune delle sue colonne sono notoriamente rumorose.

Vantaggio nel dominio. GroundOne è addestrato su RAGTruth, per cui i suoi risultati sul test set di RAGTruth e sulla colonna RAGTruth di LLM-AggreFact lo avvantaggiano rispetto a sistemi addestrati altrove; per contro, VitaminC fa parte della genealogia dei dati di addestramento di MiniCheck-RoBERTa. Ogni confronto è riportato con la relativa avvertenza, e le conclusioni poggiano sull'andamento complessivo nei tre test set anziché su uno solo.

Configurazione delle baseline. I giudici generativi sono sensibili al prompt. I loro prompt sono stati fissati prima di esaminare i risultati di test, e quello del giudice a parità di output è stato scelto sul dev set unicamente in base alla validità del formato; il parsing dell'output è stato permissivo, a loro vantaggio; il decoding non era vincolato da una grammatica, il che incide soprattutto sugli errori di formato del modello più piccolo. Non sono stati valutati né un giudice sottoposto a fine-tuning sugli stessi dati né un giudice più grande: il confronto misura dunque ciò che offre, senza addestramento, un giudice della stessa dimensione. Il System One generalista è stato eseguito con pesi a 4 bit perché il modello a 8 bit non trovava posto accanto al sistema operativo, il che potrebbe sottostimarne l'accuratezza. Il

decision model generalista d1-3B è stato eseguito zero-shot, con le domande usate per tutte le baseline System One, che potrebbero non essere la formulazione a esso più congeniale, e su un sottocampione di LLM-AggreFact.

Contaminazione. I modelli base sono stati preaddestrati su ampi corpora web che potrebbero includere parti dei test set pubblici. Poiché gli stessi modelli base, usati senza addestramento come baseline, raggiungono un'accuratezza vicina al caso sul task a tre classi, la contaminazione non può spiegare i guadagni dovuti all'addestramento; per i valori assoluti, tuttavia, non può essere esclusa del tutto.

Capitolo 8

Conclusioni e sviluppi futuri

8.1 Sintesi dei risultati

Questa tesi si è chiesta se un modello linguistico compatto, specializzato in un'unica decisione e impiegato come System One, possa verificare frase per frase, su un portatile, le risposte di un sistema RAG, e da dove provenga l'eventuale vantaggio. GroundOne, un modello Qwen2.5 sottoposto a fine-tuning che legge i verdetti di un'intera risposta in un solo forward pass, fornisce una risposta positiva, sebbene con alcune riserve.

La specializzazione funziona là dove era mirata. Sul test set di RAGTruth GroundOne raggiunge una balanced accuracy a tre classi di 0.769 con 0.5B parametri e di 0.832 con 3B, molto al di sopra degli stessi modelli non addestrati, superiore agli encoder specializzati nell'ordinare le frasi supportate e nettamente superiore a un System One generalista otto volte più grande. Su LLM-AggreFact il 3B è alla pari con il miglior encoder di piccole dimensioni, mentre su VitaminC il guadagno scompare: uno specialista non è uno strumento generale di verifica dei fatti. Un decision model generalista della stessa dimensione, addestrato a decidere su un'ampia miscela di task, conferma il compromesso: GroundOne lo supera nettamente su RAGTruth, lo eguaglia su LLM-AggreFact e resta molto indietro su VitaminC.

Il meccanismo System One ripaga su due fronti. Leggere tutti i verdetti in un solo passaggio rende la verifica delle risposte lunghe circa due volte più rapida di un giudice generativo sintetico e sei volte più rapida di uno verboso della stessa dimensione, con un output sempre valido; inoltre, restringere la decisione alle risposte ammissibili è di per sé più accurato che lasciare allo stesso modello la generazione del verdetto. A parità di output, i giudici generativi non riescono a fornire probabilità di qualità paragonabile: le loro probabilità separano a malapena le decisioni corrette da quelle errate, e quelle espresse a parole sono quasi sempre

del tipo tutto-o-niente.

Le probabilità di GroundOne si possono calibrare con una temperatura e un bias stimati sul dev set, e una soglia scelta sugli stessi dati le converte in etichette coerenti sia con le probabilità sia con la decisione valutata. L'Expected Calibration Error, da solo, non distingue però le probabilità utili da quelle inutili: la capacità discriminativa va misurata a parte.

L'intero sistema gira su un portatile da 16 GB. Una risposta lunga viene verificata in 0.41 secondi e 5.5 joule con il 0.5B, e in 2.57 secondi e 33 joule con il 3B. Il risparmio di energia rispetto a un giudice è reale, ma inferiore al risparmio di tempo, perché un singolo prefill lungo assorbe più potenza del decoding.

I limiti principali sono la ristrettezza dei dati di addestramento, per cui le contraddizioni al di fuori del data-to-text e le allucinazioni nascoste in frasi copiate vengono apprese solo in parte, e una ricetta di addestramento non del tutto robusta rispetto alla scelta del seed.

8.2 Sviluppi futuri

Da questi limiti discendono direttamente diverse linee di lavoro. La ricetta di addestramento andrebbe resa robusta abilitando la bias correction e il gradient clipping, registrando le norme dei gradienti e ripetendo i run con più seed, anche per il 3B. Dati di addestramento con contraddizioni più varie, con supporto parafrasato e con modifiche minime come quelle di VitaminC dovrebbero intervenire sia sul basso recall al di fuori del data-to-text sia sul punto cieco delle frasi copiate. Addestrare il readout sulla lettera che il modello scriverebbe spontaneamente dopo il marker potrebbe allineare readout e generazione, con effetti sull'accuratezza ancora da misurare.

Sul versante dei sistemi, per il pilastro della località restano da misurare la memoria occupata quando il verificatore opera accanto a un generatore e a un retriever, e da condurre una valutazione completa con la rete disattivata. Un confronto equo della latenza con gli encoder richiede di eseguirli in batch, mentre quello con il System One generalista richiede di interrogare entrambi i sistemi attraverso la stessa interfaccia HTTP. La verifica potrebbe inoltre procedere durante la generazione della risposta, segnalando ogni frase non appena è completa, cosa che la soluzione basata sui marker rende naturale. Infine, un assistente locale costruito sull'interfaccia del servizio permetterebbe agli utenti di scegliere la regola e la soglia più adatte ai propri documenti: è proprio qui che le probabilità calibrate mostrano il loro valore.

Appendice A

Output di ogni sistema su una stessa risposta

Questa appendice mostra che cosa restituisce ciascun sistema per una stessa risposta di T1, in modo da collegare i numeri del Capitolo 6 a output concreti. La risposta (identificativo `rt-7784`) è di tipo data-to-text: un modello linguistico l'ha scritta a partire dalla descrizione strutturata di un ristorante, ossia un record JSON con nome, indirizzo, categorie, orari di apertura (da martedì a domenica, 11:00–21:00), attributi, una valutazione media di 4.5 stelle e due recensioni di clienti. È stata scelta perché tutti i sistemi l'hanno valutata e perché contiene tutte e tre le classi, non in base alle prestazioni di un qualche sistema. Le sei frasi e le relative etichette di riferimento sono:

1. *Mesa Verde is a popular business located in Santa Barbara, CA.* — supportata.
2. *It offers a variety of services including Garage Door Services, Restaurants, Live/Raw Food, . . . , Vegan, and Salad.* — supportata.
3. *The business is open seven days a week from 11 am to 9 pm.* — contraddetta (il lunedì il locale è chiuso).
4. *Mesa Verde has a rating of 4.5 stars on Google and has received multiple positive reviews.* — non menzionata (il record non indica la fonte della valutazione e contiene due recensioni, una delle quali negativa).
5. *One customer praised the restaurant's fresh, delicious, and inventive cuisine, while another was disappointed by the taste and price of the food.* — supportata.
6. *Despite this, many customers have had positive experiences at Mesa Verde and recommend it for its fusion and vegan food options.* — supportata.

A.1 Verdetti frase per frase

La Tabella A.1 riporta il verdetto di ciascun sistema. Per GroundOne l'etichetta è quella restituita dal servizio, ricavata dalle probabilità calibrate con la soglia stimata sul dev set ($\tau = 0.935$ per il 0.5B e 0.925 per il 3B), e il numero è la P(supported) calibrata. Per gli encoder il numero è la probabilità di supporto, con la decisione presa a 0.5. S, C e N stanno per supportata, contraddetta e non menzionata; un trattino indica un errore di formato.

Tabella A.1: Verdetti sulla risposta `rt-7784`. Gli errori rispetto al riferimento sono in grassetto.

Sistema	1	2	3	4	5	6
Riferimento	S	S	C	N	S	S
GroundOne 0.5B (restituita, P_{sup})	S 0.96	S 0.98	C 0.50	N 0.05	S 0.98	S 0.97
GroundOne 3B (restituita, P_{sup})	S 0.96	S 0.99	C 0.58	N 0.21	S 0.99	**N 0.85**
B2 0.5B (token generato)	S	S	C	N	S	–
B2 3B (token generato)	S	S	C	–	S	S
0.5B non addestrato (readout)	**C**	**C**	C	**C**	**C**	**C**
Giudice sintetico 0.5B	S	S	**S**	**S**	S	S
Giudice sintetico 3B	**C**	S	**S**	**S**	**N**	S
Giudice verboso 0.5B	–	–	–	–	–	–
Giudice verboso 3B	–	–	–	–	–	–
Giudice a parità di output 3B, logit	**C**	S	**S**	**S**	**N**	S
Giudice a parità di output 3B, probabilità espresse a parole	S	**C**	**N**	**S**	**C**	**N**
MiniCheck-RoBERTa (P_{sup})	0.78	0.90	0.09	0.03	**0.43**	**0.03**
HHEM-2.1-open (P_{sup})	0.75	0.95	0.01	0.09	0.73	**0.24**
Rizzo-Flow 4B	S	S	**S**	**S**	S	S

GroundOne 0.5B classifica correttamente tutte e sei le frasi. La contraddizione nella frase 3 viene colta con una P(supported) calibrata di 0.50: ben sotto la soglia, ma lontana dalla certezza, ed è esattamente ciò che una probabilità calibrata dovrebbe esprimere per una frase corretta in tutto tranne che in un giorno della settimana. Anche GroundOne 3B coglie entrambi gli errori, ma restituisce come non menzionata la frase 6, una generalizzazione tratta dalle due recensioni: la sua P(supported) di 0.85 è inferiore alla soglia elevata imposta dal tasso di base di RAGTruth, un caso in cui un utente che preferisce meno segnalazioni abbasserebbe la soglia. A parità di pesi, il token generato (B2) dà la lettera corretta su cinque frasi su sei, ma sulla restante produce uno spazio (0.5B) o la parola "We" (3B) al posto di un verdetto. Gli encoder binari separano bene i due errori, ma respingono anche le frasi 5 o 6, che riassumono le recensioni con parole diverse; Rizzo-Flow accetta tutte le frasi. I giudici generativi falliscono nei modi quantificati nel Capitolo 6, come mostrano gli output grezzi che seguono.

A.2 Output grezzi

GroundOne 0.5B (JSON restituito, abbreviato). Un unico forward pass su 1,193 token di input, nessun token generato, 0.47 secondi (2.6 secondi per il 3B):


```
{"design": "b", "threshold": 0.935, "n_decisions": 6,
 "usage": {"input_tokens": 1193, "output_tokens": 0},
 "sentences": [
  {"index": 2, "text": "The business is open seven days a week ...",
   "scored": true, "labels": {"three": "contradicted"},
   "answers": {"three": {"supported": 0.502, "contradicted": 0.462,
                         "not_mentioned": 0.036}},
   "flag_strict": true, "flag_lenient": true},
  {"index": 3, "text": "Mesa Verde has a rating of 4.5 stars ...",
   "scored": true, "labels": {"three": "not_mentioned"},
   "answers": {"three": {"supported": 0.051, "contradicted": 0.003,
                         "not_mentioned": 0.946}},
   "flag_strict": true, "flag_lenient": false}, ...]}
```


Giudice sintetico 3B. Una riga per frase, come richiesto, ma con la lettera sbagliata in quattro frasi:

```
1: B
2: A
3: A
4: A
5: C
6: A
```

Giudice verboso 3B (primi tre verdetti). Il ragionamento riformula la frase e cita il documento a sostegno anche della frase contraddetta e di quella non supportata, e tutti i verdetti sono "A". I verdetti compaiono su una riga a sé anziché sulla stessa riga richiesta dall'istruzione, per cui il protocollo li conta come errori di formato:

```
1: Mesa Verde is located in Santa Barbara, CA, as stated in the document.
Verdict: A

2: Mesa Verde offers a variety of services including Garage Door Services,
   [...], as mentioned in the categories provided.
Verdict: A

3: The business is open seven days a week from 11 am to 9 pm, as indicated
   by the hours provided.
Verdict: A
```

Giudice verboso 0.5B. Ricopia le sei frasi, numerate, senza alcun verdetto.

Giudice a parità di output 3B, probabilità espresse a parole. Le probabilità sono tutte one-hot e, per le frasi 3 e 6, l'etichetta dichiarata (C) contraddice le probabilità dichiarate, che assegnano tutta la massa ad A:


```
{"1": {"label": "A", "p": [1.0, 0.0, 0.0]},
 "2": {"label": "B", "p": [0.0, 1.0, 0.0]},
 "3": {"label": "C", "p": [1.0, 0.0, 0.0]},
 "4": {"label": "A", "p": [1.0, 0.0, 0.0]},
 "5": {"label": "B", "p": [0.0, 1.0, 0.0]},
 "6": {"label": "C", "p": [1.0, 0.0, 0.0]}}
```


Giudice a parità di output 0.5B, probabilità espresse a parole. Lo stesso vettore per ogni frase:


```
{"1": {"label": "A", "p": [0.3, 0.3, 0.3]},
 "2": {"label": "A", "p": [0.3, 0.3, 0.3]},
 ...
 "6": {"label": "A", "p": [0.3, 0.3, 0.3]}}
```

Bibliografia

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2020 (cit. alle pp. 1, 9).

[2] Georgi Gerganov e ggml-org contributors. *llama.cpp: LLM inference in C/C++*. 2023. URL: `https://github.com/ggml-org/llama.cpp` (cit. alle pp. 1, 6, 8, 29).

[3] Apple. *MLX: An array framework for Apple silicon*. 2023. URL: `https://github.com/ml-explore/mlx` (cit. alle pp. 1, 6, 16).

[4] Cheng Niu, Yuanhao Wu, Juno Zhu, Siliang Xu et al. "RAGTruth: A Hallucination Corpus for Developing Trustworthy Retrieval-Augmented Language Models". In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. 2024 (cit. alle pp. 1, 9, 19, 20).

[5] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong et al. "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions". In: *ACM Transactions on Information Systems* (2025). DOI: `10.1145/3703155` (cit. alle pp. 1, 9).

[6] TypeSafe. *Introducing System One Models and Jev*. Blog post. 2026. URL: `https://typesafe.ai/blog/introducing-system-one-models-and-jev` (cit. alle pp. 1, 7).

[7] TheoLeeCJ. *SemIf: open option-logit System One baseline*. 2026. URL: `https://github.com/TheoLeeCJ/SemIf` (cit. alle pp. 2, 7).

[8] Simone Rizzo. *Rizzo Flow: typed decisions from an LLM, without generating a single token*. 2026. URL: `https://github.com/Rizzo-AI-Academy/rizzo-flow` (cit. alle pp. 2, 7, 29).

[9] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin et al. "Efficiently Scaling Transformer Inference". In: *Proceedings of Machine Learning and Systems (MLSys)*. 2023 (cit. alle pp. 5, 13).

[10] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah et al. “Splitwise: Efficient Generative LLM Inference Using Phase Splitting”. In: *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*. 2024 (cit. alle pp. 5, 7, 13, 45).

[11] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng et al. “Efficient Memory Management for Large Language Model Serving with PagedAttention”. In: *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*. 2023 (cit. a p. 5).

[12] Keivan Alizadeh, Iman Mirzadeh, Dmitry Belenko, Karen Khatamifard et al. “LLM in a flash: Efficient Large Language Model Inference with Limited Memory”. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. 2024 (cit. a p. 6).

[13] Apple. *MacBook Pro (14-inch, M4, 2024) – Technical Specifications*. URL: `https://support.apple.com/en-us/121552` (visitato il giorno 2026-10-10) (cit. alle pp. 6, 27).

[14] Apple. *mlx-lm: LLMs with MLX*. 2024. URL: `https://github.com/ml-explore/mlx-lm` (cit. alle pp. 6, 16).

[15] Tom Gunter, Zirui Wang, Chong Wang, Ruoming Pang et al. *Apple Intelligence Foundation Language Models*. 2024. arXiv: `2407.21075` (cit. a p. 7).

[16] Siddharth Samsi, Dan Zhao, Joseph McDonald, Baolin Li et al. “From Words to Watts: Benchmarking the Energy Costs of Large Language Model Inference”. In: *IEEE High Performance Extreme Computing Conference (HPEC)*. 2023 (cit. alle pp. 7, 45).

[17] Alexandra Sasha Luccioni, Yacine Jernite e Emma Strubell. “Power Hungry Processing: Watts Driving the Cost of AI Deployment?” In: *Proceedings of the ACM Conference on Fairness, Accountability, and Transparency (FAccT)*. 2024 (cit. alle pp. 7, 45).

[18] Jacky Kwok, Hangoo Kang, Tarun Suresh, Jon Saad-Falcon, Marco Pavone, Christopher Ré e Azalia Mirhoseini. *Contrastive Language Models: A System One Model for Fast and Generalizable Decision-Making*. Blog post, not peer-reviewed. 2026. URL: `https://contrastive-lm.notion.site` (cit. a p. 7).

[19] Convai Innovations. *Laya: a System One decision model (model card, convaiinnovations/laya)*. URL: `https://huggingface.co/convaiinnovations/laya` (visitato il giorno 2026-10-10) (cit. a p. 8).

[20] Seyedarmin Azizi, Erfan Baghaei Potraghloo e Massoud Pedram. *Labels Override Definitions in Jev-Style Typed Decision Models*. 2026. arXiv: `2610.02586` (cit. a p. 8).

[21] Tobias Deußer, Lorenz Sparrenberg e Rafet Sifa. *Evaluating and Benchmarking the System One Model Jev*. 2026. arXiv: 2609.37647 (cit. alle pp. 8, 38).

[22] Gowthamkumar Nandakishore. *Laya as a Typed Probabilistic Assessor: An Independent Reproduction and a Preregistered Study of Calibration and Selective Escalation*. 2026. arXiv: 2609.33843 (cit. a p. 8).

[23] Cho Yin Yong. *DecideBench (dataset card, choyiny/decidebench)*. URL: https://huggingface.co/datasets/choyiny/decidebench (visitato il giorno 2026-10-10) (cit. a p. 8).

[24] Hugging Face. *Models tagged decision-model*. URL: https://huggingface.co/models?other=decision-model (visitato il giorno 2026-10-10) (cit. a p. 8).

[25] Cloudflare. *Clef: open decision models (model card, Cloudflare/clef)*. URL: https://huggingface.co/Cloudflare/clef (visitato il giorno 2026-10-10) (cit. a p. 8).

[26] Liquid AI. *d1-3B: a decision model built on LFM2.5-VL-3B (model card, LiquidAI/d1-3B, revision 051bcc4)*. URL: https://huggingface.co/LiquidAI/d1-3B (visitato il giorno 2026-10-10) (cit. alle pp. 8, 29).

[27] Perplexity. *pplx-decider-v1.1-27b (model card)*. URL: https://huggingface.co/perplexity-ai/pplx-decider-v1.1-27b (visitato il giorno 2026-10-10) (cit. a p. 8).

[28] multimodalart. *Jev Decision Index: a leaderboard of System One decision models (Hugging Face Space, v0.2.1)*. URL: https://huggingface.co/spaces/multimodalart/jev-decision-index (visitato il giorno 2026-10-10) (cit. alle pp. 8, 36).

[29] Ruoqi Guo, Yi Liu, Gelei Deng, Yuekang Li et al. *Just Ask Jev: Reinforcement Learning for Calibrated Decisions as a Zero-Shot Detector of AI Alignment Failures*. 2026. arXiv: 2609.29429 (cit. a p. 8).

[30] Yubo Li, Yidi Miao, Ramayya Krishnan e Rema Padman. *JEV-as-a-Judge: Accept When Confident, Escalate When Unsure*. 2026. arXiv: 2609.26550 (cit. a p. 8).

[31] Xinpeng Wang, Bolei Ma, Chengzhi Hu, Leon Weber-Genzel et al. ""My Answer is C": First-Token Probabilities Do Not Match Text Answers in Instruction-Tuned Language Models". In: *Findings of the Association for Computational Linguistics: ACL 2024*. 2024. arXiv: 2402.14499 (cit. alle pp. 8, 14).

[32] Chujie Zheng, Hao Zhou, Fandong Meng, Jie Zhou e Minlie Huang. "Large Language Models Are Not Robust Multiple Choice Selectors". In: *International Conference on Learning Representations (ICLR)*. 2024 (cit. alle pp. 9, 14).

[33] Adina Williams, Nikita Nangia e Samuel R. Bowman. "A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference". In: *Proceedings of NAACL-HLT 2018*. 2018 (cit. a p. 9).

[34] Yixin Nie, Adina Williams, Emily Dinan, Mohit Bansal, Jason Weston e Douwe Kiela. "Adversarial NLI: A New Benchmark for Natural Language Understanding". In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2020 (cit. alle pp. 9, 20).

[35] Tal Schuster, Adam Fisch e Regina Barzilay. "Get Your Vitamin C! Robust Fact Verification with Contrastive Evidence". In: *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 2021, pp. 624–643. DOI: 10.18653/v1/2021.naacl-main.52. URL: https://aclanthology.org/2021.naacl-main.52 (cit. alle pp. 9, 19, 20).

[36] Philippe Laban, Tobias Schnabel, Paul N. Bennett e Marti A. Hearst. "SummaC: Re-Visiting NLI-based Models for Inconsistency Detection in Summarization". In: *Transactions of the Association for Computational Linguistics* 10 (2022), pp. 163–177 (cit. a p. 9).

[37] Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis et al. "FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation". In: *Proceedings of EMNLP 2023*. 2023 (cit. a p. 9).

[38] Yuheng Zha, Yichi Yang, Ruichen Li e Zhiting Hu. "AlignScore: Evaluating Factual Consistency with a Unified Alignment Function". In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*. 2023 (cit. alle pp. 10, 39).

[39] Liyan Tang, Philippe Laban e Greg Durrett. "MiniCheck: Efficient Fact-Checking of LLMs on Grounding Documents". In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2024 (cit. alle pp. 10, 19, 20, 30).

[40] Vectara. *HHEM-2.1-Open: Hughes Hallucination Evaluation Model*. 2024. URL: https://huggingface.co/vectara/hallucination_evaluation_model (cit. alle pp. 10, 30).

[41] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang et al. "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena". In: *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. 2023 (cit. a p. 10).

[42] Shahul Es, Jithin James, Luis Espinosa-Anke e Steven Schockaert. "RAGAs: Automated Evaluation of Retrieval Augmented Generation". In: *Proceedings of EACL 2024: System Demonstrations*. 2024 (cit. a p. 10).

[43] Jon Saad-Falcon, Omar Khattab, Christopher Potts e Matei Zaharia. "ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems". In: *Proceedings of NAACL 2024*. 2024 (cit. a p. 10).

[44] Selvan Sunitha Ravi, Bartosz Mielczarek, Anand Kannappan, Douwe Kiela e Rebecca Qian. *Lynx: An Open Source Hallucination Evaluation Model*. 2024. arXiv: 2407.08488 (cit. a p. 10).

[45] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil e Hannaneh Hajishirzi. "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection". In: *International Conference on Learning Representations (ICLR)*. 2024 (cit. a p. 10).

[46] Brandon T. Willard e Rémi Louf. *Efficient Guided Generation for Large Language Models*. 2023. arXiv: 2307.09702 (cit. alle pp. 10, 29).

[47] Liyan Tang, Philippe Laban e Greg Durrett. *LLM-AggreFact Leaderboard*. 2024. URL: https://llm-aggrefact.github.io/ (visitato il giorno 2026-10-05) (cit. alle pp. 10, 19, 20).

[48] Chuan Guo, Geoff Pleiss, Yu Sun e Kilian Q. Weinberger. "On Calibration of Modern Neural Networks". In: *International Conference on Machine Learning (ICML)*. 2017 (cit. alle pp. 10, 15).

[49] Mahdi Pakdaman Naeini, Gregory F. Cooper e Milos Hauskrecht. "Obtaining Well Calibrated Probabilities Using Bayesian Binning". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. 2015 (cit. alle pp. 10, 15).

[50] Tony Z. Zhao, Eric Wallace, Shi Feng, Dan Klein e Sameer Singh. "Calibrate Before Use: Improving Few-Shot Performance of Language Models". In: *International Conference on Machine Learning (ICML)*. 2021 (cit. alle pp. 10, 15).

[51] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan et al. *Language Models (Mostly) Know What They Know*. 2022. arXiv: 2207.05221 (cit. a p. 10).

[52] Katherine Tian, Eric Mitchell, Allan Zhou, Archit Sharma et al. "Just Ask for Calibration: Strategies for Eliciting Calibrated Confidence Scores from Language Models Fine-Tuned with Human Feedback". In: *Proceedings of EMNLP 2023*. 2023 (cit. alle pp. 10, 43).

[53] Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li et al. "Can LLMs Express Their Uncertainty? An Empirical Evaluation of Confidence Elicitation in LLMs". In: *International Conference on Learning Representations (ICLR)*. 2024 (cit. alle pp. 10, 43).

[54] Yonatan Geifman e Ran El-Yaniv. "Selective Classification for Deep Neural Networks". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017 (cit. alle pp. 10, 30).

[55] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang e Weizhu Chen. "LoRA: Low-Rank Adaptation of Large Language Models". In: *International Conference on Learning Representations (ICLR)*. 2022 (cit. alle pp. 11, 14).

[56] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman e Luke Zettlemoyer. "QLoRA: Efficient Finetuning of Quantized LLMs". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2023 (cit. alle pp. 11, 14).

[57] Qwen Team. *Qwen2.5 Technical Report*. 2024. arXiv: 2412.15115 (cit. alle pp. 11, 14, 27).

[58] Jesse Dodge, Gabriel Ilharco, Roy Schwartz, Ali Farhadi, Hannaneh Hajishirzi e Noah Smith. *Fine-Tuning Pretrained Language Models: Weight Initializations, Data Orders, and Early Stopping*. 2020. arXiv: 2002.06305 (cit. alle pp. 11, 28).

[59] Marius Mosbach, Maksym Andriushchenko e Dietrich Klakow. "On the Stability of Fine-tuning BERT: Misconceptions, Explanations, and Strong Baselines". In: *International Conference on Learning Representations (ICLR)*. 2021 (cit. alle pp. 11, 35, 49).

[60] Razvan Pascanu, Tomas Mikolov e Yoshua Bengio. "On the Difficulty of Training Recurrent Neural Networks". In: *International Conference on Machine Learning (ICML)*. 2013 (cit. alle pp. 11, 35, 49).

[61] Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder e Li Deng. *MS MARCO: A Human Generated MAchine Reading COmprehension Dataset*. CoCo Workshop at NIPS 2016. 2016. arXiv: 1611.09268 (cit. a p. 19).

[62] Karl Moritz Hermann, Tomáš Kočiský, Edward Grefenstette, Lasse Espeholt, Will Kay, Mustafa Suleyman e Phil Blunsom. "Teaching Machines to Read and Comprehend". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2015 (cit. a p. 19).

[63] Abigail See, Peter J. Liu e Christopher D. Manning. "Get To The Point: Summarization with Pointer-Generator Networks". In: *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2017 (cit. a p. 19).

[64] Yelp. *Yelp Open Dataset*. URL: https://business.yelp.com/data/resources/open-dataset/ (visitato il giorno 2026-10-05) (cit. a p. 19).

[65] Liyan Tang, Tanya Goyal, Alex Fabbri, Philippe Laban, Jiacheng Xu, Semih Yavuz, Wojciech Kryściński, Justin Rousseau e Greg Durrett. "Understanding Factual Errors in Summarization: Errors, Summarizers, Datasets, Error Detectors". In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*. 2023 (cit. a p. 20).

[66] Liyan Tang, Igor Shalyminov, Amy Wing-mei Wong et al. "TofuEval: Evaluating Hallucinations of LLMs on Topic-Focused Dialogue Summarization". In: *Proceedings of NAACL 2024*. 2024 (cit. a p. 20).

[67] Nelson F. Liu, Tianyi Zhang e Percy Liang. "Evaluating Verifiability in Generative Search Engines". In: *Findings of EMNLP 2023*. 2023 (cit. a p. 20).

[68] Chaitanya Malaviya, Subin Lee, Sihao Chen, Elizabeth Sieber, Mark Yatskar e Dan Roth. "ExpertQA: Expert-Curated Questions and Attributed Answers". In: *Proceedings of NAACL 2024*. 2024 (cit. a p. 20).

[69] Hung-Ting Chen, Fangyuan Xu, Shane Arora e Eunsol Choi. "Understanding Retrieval Augmentation for Long-Form Question Answering". In: *Conference on Language Modeling (COLM)*. 2024 (cit. a p. 20).

[70] Ryo Kamoi, Tanya Goyal, Juan Diego Rodriguez e Greg Durrett. "WiCE: Real-World Entailment for Claims in Wikipedia". In: *Proceedings of EMNLP 2023*. 2023 (cit. a p. 20).

[71] Alon Jacovi, Yonatan Bitton, Bernd Bohnet, Jonathan Herzig, Or Honovich, Michael Tseng, Michael Collins, Roee Aharoni e Mor Geva. "A Chain-of-Thought Is as Strong as Its Weakest Link: A Benchmark for Verifiers of Reasoning Chains". In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. 2024 (cit. a p. 20).

[72] Yuxia Wang, Revanth Gangi Reddy, Zain Muhammad Mujahid et al. "Factcheck-Bench: Fine-Grained Evaluation Benchmark for Automatic Fact-checkers". In: *Findings of EMNLP 2024*. 2024 (cit. a p. 20).

[73] Liyan Tang. *LLM-AggreFact dataset card (lytang/LLM-AggreFact)*. URL: `https://huggingface.co/datasets/lytang/LLM-AggreFact` (visitato il giorno 2026-10-05) (cit. a p. 20).

[74] Tal Schuster. *VitaminC dataset card (tals/vitaminc)*. URL: `https://huggingface.co/datasets/tals/vitaminc` (visitato il giorno 2026-10-06) (cit. a p. 20).

[75] Nipun Sadvilkar e Mark Neumann. "PySBD: Pragmatic Sentence Boundary Disambiguation". In: *Proceedings of the Second Workshop for NLP Open Source Software (NLP-OSS)*. 2020, pp. 110–114. DOI: `10.18653/v1/2020.nlposs-1.15`. URL: `https://aclanthology.org/2020.nlposs-1.15` (cit. a p. 21).

[76] Boxi Yu, Yuzhong Zhang, Liting Lin, Lionel Briand e Emir Muñoz. *Retromorphic Testing with Hierarchical Verification for Hallucination Detection in RAG*. 2026. arXiv: 2603.27752. URL: https://arxiv.org/abs/2603.27752 (cit. alle pp. 23, 50).

[77] Philipp Koehn. "Statistical Significance Tests for Machine Translation Evaluation". In: *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2004, pp. 388–395. URL: https://aclanthology.org/W04-3250 (cit. a p. 31).

[78] Taylor Berg-Kirkpatrick, David Burkett e Dan Klein. "An Empirical Investigation of Statistical Significance in NLP". In: *Proceedings of EMNLP-CoNLL 2012*. 2012, pp. 995–1005. URL: https://aclanthology.org/D12-1091 (cit. a p. 31).

[79] Rotem Dror, Gili Baumer, Segev Shlomov e Roi Reichart. "The Hitchhiker's Guide to Testing Statistical Significance in Natural Language Processing". In: *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2018, pp. 1383–1392. URL: https://aclanthology.org/P18-1128 (cit. a p. 31).